

Classifying Quantum Programming Languages

JACOB J. B. TREVOR, School of Computing Science, University of Glasgow, United Kingdom

There has been an explosion of activity in quantum programming language research over the last two decades. The most advanced languages are capable of providing useful compile-time guarantees and managing qubit use automatically. Approaches vary from high-level type systems research to low level assembly design. We aim to provide a high-level overview of trends in the field by investigating a number of significant state-of-the-art quantum programming languages. Our key contributions include a summary of common runtime models and compiler architectures, a new scheme for classifying quantum programming languages by their expressiveness based on control flow, and an analysis of language allocation strategies and execution environments. Whenever possible, we provide source code samples of well-known algorithms as a systematic basis of comparison.

CCS Concepts: • **Theory of computation** → **Program semantics**; **Quantum information theory**.

ACM Reference Format:

Jacob J. B. Trevor. 2026. Classifying Quantum Programming Languages. *ACM Trans. Quantum Comput.* 1, 1 (June 2026), 53 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

In the 20 years since the last substantive, broad-based survey of quantum programming language (QPL) research [17], many aspects of the field have changed. Although commodity quantum computers may still seem unrealistic, vast strides have been made in the design and implementation of quantum programming languages. The focus has shifted away from abstract semantics as industry has become more involved with the development of hardware, and correspondingly the software required to operate it. The application of concepts from high-level language design has enabled, for instance, static verification of the safe use of qubits [6], automatic uncomputation of garbage qubits [34], and conversion of classical functions into quantum oracles [47]. There are several competing Quantum Intermediate Representations (IRs) [3, 9, 10, 26, 46], all of which have been devised in the last decade. These IRs facilitate novel optimisation techniques for quantum algorithms [26], providing compilation targets for high-level languages, and a basis for the evaluation of compiler frontends and quantum algorithms.

In this paper, we investigate the current state-of-the-art in QPL research by reviewing a representative selection of modern quantum programming languages. The languages were chosen for their significance — they present important novel ideas, or were highly influential on later work. As a basis for comparison, we provide idiomatic implementations of a variety of standard quantum programs which demonstrate interesting or useful behaviours.

We will first explore a general background in quantum computing (§2). We will enumerate the unique challenges presented by quantum computing to the programmer that can be mitigated by well designed programming languages (§2.2). In the process, we will present a novel framework for understanding the expressiveness of quantum programming languages, in terms of the degree

Author's Contact Information: Jacob J. B. Trevor, School of Computing Science, University of Glasgow, Glasgow, United Kingdom, j.trevor.1@research.gla.ac.uk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2643-6817/2026/6-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

of control flow they allow (§2.2.4). We will then move on to a survey of low-level languages (§3), and then of high-level languages (§4). For each language, we will consider which of the challenges have been addressed and how, as well as how well these solutions are executed. More specifically: are they proof-of-concept artefacts produced in the course of research? Are they ready for use at an industrial scale? We will then present a summary of other related work (§5). Finally, we will present our conclusions: which problems remain unaddressed? Thus, what spaces in the landscape remain for future study (§6)?

2 Background

We will assume moderate familiarity with the basic ideas of quantum computing: qubits and gates, their representation as matrices and vectors, and circuits. For those unfamiliar, we provide an introduction to the topic in §A, which contains all the details necessary to understand the rest of the paper. If necessary, we encourage readers to go over §A first, before returning to this point. A complete background in quantum computing can be found in Nielsen and Chuang [32].

2.1 A word on linearity

In this paper (and in the field of quantum programming languages more generally) “linear” can have two meanings. The first is linear in the sense of *linear algebra*. Let K be a field, and V and W be vector spaces over K . A transformation $f : V \rightarrow W$ is linear if it observes the two following properties:

$$\begin{array}{ll} \forall a, b \in V, & \forall a \in V, \forall \lambda \in K, \\ f(a + b) = f(a) + f(b) & \lambda f(a) = f(\lambda a) \end{array}$$

Additivity
Scalar multiplication

We call this kind of linearity *algebraic linearity*. Algebraic linearity underpins the standard presentation of quantum information theory; quantum gates are algebraically linear transformations.

The second meaning of linear is the one more common in programming language research. A resource is linear if it must be used exactly once. This concept was introduced by Girard with his linear logic [18, 19] — hence the name *logical linearity*. These ideas were developed further into the ideas of linear types [53]. In classical computing, this can be used to e.g. enforce that a file handle cannot be written to after it is closed.

A consequence of the fact that quantum gates must always be unitary (and therefore reversible) is that quantum information is logically linear; it cannot be copied, nor thrown away. The no cloning theorem [56] states that we cannot have a transformation that gives:

$$|0\rangle \otimes |\phi\rangle \Rightarrow |\phi\rangle \otimes |\phi\rangle$$

i.e. returns two *independent* copies of $|\phi\rangle$. Note that this elucidates two properties. Firstly, we cannot *throw away* the information on the first qubit. Secondly, we cannot *copy* the information on the second. Thus the use of the information that a given qubit represents must be linear.

2.2 Challenges of QPL

When designing QPL, there are principally four behaviours that benefit from special treatment. We will discuss each in detail. They are:

- non-locality
- logical linearity
- reversibility

Listing 1. Non-locality

```

fn non_local() {
  a, b <- fresh
  H a
  CX a, b
  M a
}

```

Listing 2. Bell pair allocator

```

fn bell_pair () {
  // construct and return a  $|\phi^+\rangle$  state
  a, b <- fresh
  H a
  CX a b
  return (a, b)
}

```

- classical control flow

2.2.1 Non-locality. Entanglement introduces non-local behaviours to the program. Measuring one qubit in an entangled state may impact all the others. Consider the example in Listing 1, which constructs a Bell state ($|\phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$) and measures one of its component qubits. Here, even though we only measure a , the value of b is also collapsed. Thus, for certain operations, the behaviour can extend beyond the information present at the call site.

This is difficult to reason about for a programmer — finding way to mitigate this is of interest. However, it also causes issues for the language designer. Most obviously, problems arise in memory management; there are broadly three approaches, which we will present in order of increasing complexity.

- (1) **Static allocation** The simplest approach, wherein all qubits are allocated at the beginning of the program.
- (2) **Stack-based allocation** Allocation follows a nested scope structure (traditionally, function calls). The key characteristic of stack-based allocation is that a qubit can never outlive the scope in which it was defined.
- (3) **Dynamic allocation** Qubits can be allocated and deallocated at any time, as appropriate for their lifetimes.

The allocation strategy of a language has implications for expressiveness, for the ergonomics of the language, and for the complexity of any associated runtime. For instance, languages with only static allocation cannot be Turing complete, and only dynamic allocation can allow users to define their own allocators — a desirable example of which is illustrated in Listing 2.

The principal challenge is handling qubit de-allocation in a safe manner; freeing a qubit should not result in data corruption, or any other surprising adverse effect. Listing 3 presents a simple example which breaks a naïve stack implementation. Note that there is no safe way to free b after the end of `foo`. A region-based approach [51] can solve the issue (e.g. as suggested by Peduri et al. [36]), by placing a and b in the same region. However, this is significantly more complex than a naïve stack implementation.

Listing 3. Example program that is not safe under stack-based memory management

```

fn foo(a: Qubit) {
  b <- fresh
  H b
  CX b a
}

fn main() {
  a <- fresh
  foo (a)
  M a
}

```

Listing 4. A program that is only valid under reference semantics

```

fn refOnly() {
  a <- fresh
  H a
  H a
}

```

Part of the difficulty is that entanglement *may* result from a given interaction, but it does not have to. Consider for instance applying CNOT across two separable qubits. In the general case, they will become entangled. However, this is not the case if the control bit is in a basis state. Determining which qubits are entangled at some particular stage of the computation is known as *entanglement analysis* [37, 38, 41, 58]. Generally, good entanglement analyses are expensive, requiring a form of symbolic execution.

2.2.2 Logical Linearity. Quantum information is a logically linear resource — it must be consumed exactly once. This can make qubits harder to reason about, depending on the language and the semantics of its names.

There is no issue in languages with reference semantics — i.e. settings where variable names refer to the qubit, rather than the information it holds. A program which references the same physical qubit twice may be perfectly correct. Gate applications should be understood to be mutating the qubit they are applied to. Under reference semantics, this program in Listing 4 is valid.

However, in languages with value semantics — where names refer to a quantum state, rather than the qubit the state is held on — problems can arise. Languages with value semantics do not permit programs like Listing 4. Gate applications *consume* the input, and yield a new output. The name refers to a quantum state, which no longer exists after a gate is applied to it.

These languages must somehow enforce logical linearity to provide guarantees that the programs they describe are physical. The most common approach to this is a linear type system. These are well understood, but add a degree of complexity that designers and programmers must account for.

2.2.3 Reversibility. All quantum operations must be reversible. The first interesting idea this gives rise to is *automatic inverse generation*. Undoing an operation is very common in quantum computing. For example, when computing an ancillary value, we often want to safely dispose of it without collapsing the useful state of the program. This requires us to uncompute the qubit — which

means applying the inverse of every operation it underwent, so that it returns to its initial starting condition.

Without automatic inverse generation, this kind of uncomputation essentially requires the programmer to maintain two copies of the relevant code — one in the forward direction and one in reverse. Removing this requirement by generating inverse gates automatically is a much more robust pattern, that lessens the load on the programmer.

The second interesting idea to discuss is automatic oracle generation [47]. In this context, a *quantum oracle* is a classical function, implemented to operate over qubits rather than bits. This is critical for example in the implementation of Grover’s algorithm [22]. Very often, the function we wish to describe is simply a classical function on bits, where it is trivial to write an implementation in an ordinary classical language.

The idea of automatic oracle generation is to find ways to automatically generate a quantum implementation of such classical functions. This is trickier than it may at first sound; classical operations are not inherently reversible (consider $\text{True OR } x$), and so extra work is required to keep track of information that would otherwise be thrown away in a typical classical implementation. Doing this efficiently is of specific interest, for example doing so with the minimal number of qubits [24].

2.2.4 Classical Control Flow. Integrating classical control flow (CCF) structures into quantum programming languages requires special consideration. Historically, the desire for “flat” programs — programs with no runtime control flow — has been common. Early approaches to quantum IRs [3] expended much effort to maintain a circuit model, and thus generated code without control flow.

However, flat code is not as expressive as non-flat code. In fact, languages can be placed in an expressiveness hierarchy based on the level of classical control flow they permit. This hierarchy is illustrated in Figure 1.

At the bottom of the hierarchy (**Pt. F**) one will find flat programs, or programs in which all CCF is statically evaluated. Almost no high-level QPLs are truly flat — even the most restricted languages tend to allow static (i.e. compiler resolved) CCF for the convenience of the programmer. They will, however, compile down to flat code, without any CCF. Thus, static code is not any more expressive than flat code — it is simply more compact. Many early languages inhabit this point in the lattice — Scaffold and its QASMs (discussed later in §3.1), for instance.

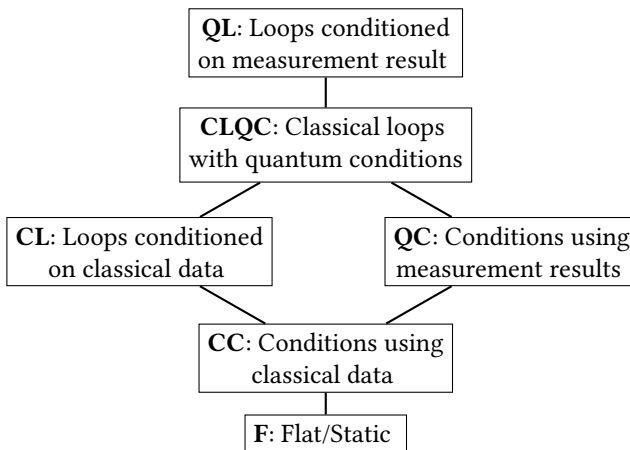


Fig. 1. Expressiveness hierarchy of CCF in QPL

The next point in the lattice (Pt. CC) introduces some runtime conditions. The key difference is that sections of code can be conditioned on a classical value, known only at runtime. The same program can be expressed in a static program if the value is known at compile time. It's also important to note that this kind of language does not allow returning to an earlier point in the program — so there are no looping structures. Similarly, we also do not allow control flow to be conditioned on the result of a measurement.

These two extensions lead us on to the next two points in the lattice. Adding runtime loops (Pt. CL) allows for programs to return to an earlier point. This is the lowest point in the lattice where we find quantum programs that may not terminate.

Dynamic conditions with measurement results (Pt. QC) allow us to utilise a measurement result as the condition for a gate application.

Joining these two features together, we reach Pt. CLQC. This point allows runtime loops and conditions with measurement results. At this point, loops must be conditioned only on classical values.

Most expressive of all is dynamic looping with feedback (Pt. QL). Here, we allow looping behaviour to be conditioned on the result of a quantum measurement. It is not difficult to devise even simple programs that require at least this level of expressiveness. For example, consider Listing 5.

Listing 5. Geometric distribution sampler¹

```
// @QL

// Sample the geometric distribution
// repeated runs will produce results whose probability
// distribution is a geometric distribution.

fn geometric() {
  count = 0
  x <- fresh
  do {
    H x
    res = M x
    count += 1
  } while (res)
  return count
}
```

2.3 Runtime Models

We now consider the standard conceptual models of quantum computers from the literature.

Circuits naturally represent *flat codes* — those without any control flow, at Pt. F. Circuits are particularly common in algorithmics, wherein the “question” is often framed as “How do we produce a circuit which does X?”. Even iterative algorithms are often described in this manner: for instance, Nielsen and Chuang’s presentation [32] of Grover’s algorithm [22]. This is despite the fact that a more natural presentation may be as a proper iterative code.

In practise, this “pure” circuit model is very rare, precisely because it is not particularly expressive. Generally, we allow some non-flat behaviours — particularly, classical conditions. Generally, when

¹the Silq website <https://silq.ethz.ch/documentation>

we refer to the “circuit runtime model”, this model — extended with classically-conditioned gate applications — is what we mean. This model is at [Pt. CC](#), or at [Pt. QC](#) when measurement is allowed.

A more nuanced model of quantum computing was presented by Knill [29]. The *QRAM model* is notable in that it is not a circuit. Instead, we are to imagine we have access to some large amount of quantum memory (in the form of *quantum registers*), to which we can apply unitary operators or measurement. This can be interleaved with arbitrary classical computation. This interleaving allows for a richer programming model — QRAM programs are not, by nature, flat. The QRAM model is fully expressive, and permits programs at [Pt. QL](#).

In his pseudocode, Knill takes care to unify the representation of quantum and classical data; both are represented by registers. Syntactically, quantum registers are underlined, so that readers can understand that some superposition or entanglement may be present. Semantically, however, either class of registers should be considered special cases of generalised “registers”. The model was originally developed to provide a more flexible and natural basis for describing quantum algorithms, but it has been widely adopted within QPL research as a mental model for the quantum computer.

Building on top of QRAM, we might also consider the *coprocessor model*. The coprocessor model assumes further details of the quantum computer. In this model, a quantum computer consists of a scheduler (a conventional classical computer), a quantum processing unit (QPU), and some amount of quantum memory. This model was first presented by Oskin et al. [33]. Their presentation is concerned with how to design hardware for error correction, but it has also been highly influential in programming language research. It is implicitly assumed by any compiler or interpreter which uses a dual-stage pipeline.

2.3.1 Compiler Architectures. In a dual-stage compiler, program runtime is split into two stages. First, some portion of the code runs on a classical computer, to generate the set of instructions that are to be run on the quantum computer. This is referred to as *circuit generation*. Then, the instructions are executed on the actual quantum computer. This is referred to as *circuit execution*.

In a compiled language (e.g. Quipper/Haskell [21]), this ends up being a three-stage process. The program is first compiled to a binary by the compiler. When the program is executed, it then goes through the circuit generation phase, and then circuit execution.

These two phases correspond to the scheduler and QPU components of the coprocessor model. This creates a strong distinction between the classical and quantum components of the program, which can make it difficult to discuss *mixed programs*. Mixed (quantum-classical) programs are those which involve classical and quantum computing components. Many programs (indeed, we argue, many problems) are inherently mixed, requiring the interleaving of quantum and classical components. A lack of support for mixed computing was identified at an early stage by Gay [17], and a key finding of our work will be to conclude that the gap has not (yet) been properly filled.

In the dual-stage architecture, in order to express points in the lattice at [Pt. QC](#) or higher, we need a way to convert a circuit-execution-time value into one at circuit generation time. In the literature, this is often known as “dynamic lifting” [14, 35].

Part of the motivation for the dual-stage architecture is to provide a way to model the instability of qubits. Qubits have a limited *coherence time*: they can only retain a state for some amount of time, before it is degraded by noise. This stage of development in quantum computing is referred to as the “noisy intermediate scale quantum” (NISQ) model of quantum computers, first put forward by Preskill [40]. The precise characteristics of coherence time are important. If the coherence time is very short, then it may not be possible to interleave quantum and classical components. This is the nuance which dual-stage compilers attempt to capture, and why dynamic lifting is usually considered an expensive operation.

The stratification of execution into these stages, and the existence of dynamic lifting, make discussing expressiveness more complex. Should we be considering the expressiveness of the whole binary? Or only the component which runs on the quantum computer? Should inputs to the circuit generator be considered runtime values? Or should that be reserved for values in the circuit execution phase? Answering these questions is critical for establishing the expressiveness of the language. Additionally, for some programs moving freely between the two stages produces much cleaner and more understandable code. In our view, this calls into question the value of the distinction — at least when it comes to programmer facing languages, as opposed to quantum IRs.

Another kind of quantum compiler — the single-stage compiler — does not distinguish between circuit generator or execution. There is a single compile time, in which a binary is produced. This binary is intended to encompass the entire program. This architecture is standard in classical computing, so we will not discuss it much here. It is notable in quantum computing mostly by way of contrast with the dual stage architecture.

These models all fall under the broad umbrella of “quantum data, classical control” (QDCC), a slogan coined by Selinger in 2004 [45]. This was formalised by Perdrix [39], with his “classically-controlled quantum Turing machine” (CQTM). A key result of that paper is to show how CQTM can subsume both circuits and classical computing, without a loss of efficiency.

2.4 Example Programs

For each point on the CCF lattice (barring **Pt. CLQC**), we have produced an example program, which requires at least as much expressiveness. These are:

- **Pt. F** Deutsch’s Algorithm [12] — Determines whether a function $f : \mathbb{B} \rightarrow \mathbb{B}$ is balanced or constant, notable as the first algorithm to demonstrate quantum supremacy.
- **Pt. CC** Parametric Deutsch’s Algorithm — The same as above, but the choice of function f is determined at runtime.
- **Pt. QC** Quantum teleportation [5] — Transfer a quantum state from one qubit to another, possibly over some arbitrary distance.
- **Pt. CL** n -bit-rand — Sample an n -bit random number, using a $|+\rangle$ state repeatedly. n is determined at runtime.
- **Pt. QL** Geometric — Sample the geometric distribution.

For brevity, the implementations will be largely left out of the main body of the paper, though some samples will be included when relevant. A pseudocode presentation can be found in §B, along with a more detailed explanation in the style of the one we are about to present for quantum teleportation. Implementations in other languages can be found in §C. As well as providing a convenient way to explore expressiveness, these examples also serve as a standardised set of programs to compare across languages. This allows us to judge (in some subjective, heuristic way) how elegant each language was.

2.4.1 Quantum Teleportation. Quantum teleportation has been called the “hello world” of quantum computing². It is one of the simplest interesting programs one can write involving qubits. In quantum teleportation, a qubit of information is transferred from one qubit to another, potentially over some arbitrary distance.

First, we produce a pair of entangled qubits (in the Bell $|\phi^+\rangle$ state). One of these is retained by the receiver, the other is sent to the sender. The message and sender qubits are then entangled, and a Hadamard gate is then applied to the message qubit.

²<https://www.youtube.com/watch?v=v7b4J2INq9c>

A two-bit measurement is then performed on the message and the sender. This provides sufficient information to fully reconstruct the state of the message at the receiver, by conditionally applying a Z and X gate. Fig. 2 shows a circuit diagram for quantum teleportation.

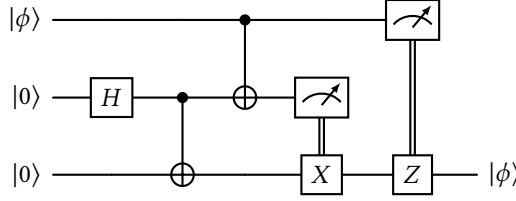


Fig. 2. Circuit diagram for quantum teleportation

Within this protocol, there are several interesting features. Firstly, quantum teleportation requires a language with at least the expressiveness of **Pt. QC**. That is, using a classical measurement result to condition some continuation. Some languages do not support this, and so cannot represent quantum teleportation, despite its relative simplicity.

The protocol also requires us to generating a Bell pair of entangled qubits. This is a very common procedure, and so we may wish to extract the behaviour to a function. However, encapsulating this behaviour requires a fully dynamic memory model. As we will see, this is quite rare among high-level QPLs.

3 Low-Level Approaches

Low-level approaches to programming languages are concerned with code generation, in one way or another. Thus, central to all low-level QPL methods are quantum assemblies and quantum IRs. Though “quantum assembly” (QASM) is the dominant term, “quantum IR” (QIR) is more accurate to the real aims of such projects. In general, quantum IRs cannot run directly on real quantum hardware.

Instead, the aim is to provide a target for quantum compilers which could be later transformed to a format suitable for execution on a specific QC. We can still use the IR form in the present — either as an optimisation start/end point, as the input to a simulator, or for evaluation purposes. QIRs can be used to evaluate QPL research, but also to estimate resource usage and validate algorithms. They can also be used to develop generalised tools (such as compiler passes for inserting quantum error correction).

IRs in classical computing form the meeting point of three distinct compiler components: frontends, backends, and optimisers. When designing an IR, it is therefore important to consider the aims of each of these three components, and how best to support them. A particular IR need not cater to the needs of all three: The MLIR project [30] was designed to provide a “multi-layer IR” — really, multiple IRs which can easily be transformed into each other, which support different goals (generally, different kinds of optimisations).

What might we want in our QIRs? Obviously, it’s important that they are *expressive*. Because they form the lowest point in the compilation pipeline, limitations at this level are bubbled up to earlier steps. Thus, we will consider which point on the CCF lattice these languages occupy. However, expressiveness goes beyond just this. At higher points in the lattice, mixed computation becomes more useful. It’s important that our IRs are able to express this, for practical reasons. Classical computation is far cheaper and easier than quantum equivalents. Thus, we need to be able to reason about them together.

Listing 6. Broken Scaffold code

```

int main() {
    qbit q;
    cbit c;

    H ( q );
    c = MeasZ ( q );

    return 0;
}

```

For each language we consider, we will discuss the defining characteristics and features, as well as any notable issues or problems left for future work. We will briefly touch on the handling of qubit allocation. Finally, we will discuss what execution environments each supports.

3.1 Scaffold/ScaffCC

Among the earliest attempts at a quantum assembly is presented in ScaffCC [27]. This paper presents a compiler for a quantum language embedded in C, and of particular interest to us, a family of quantum IRs.

ScaffCC’s “QASM” and its variants are flat codes (Pt. F) with stack-based allocation, largely inspired by the work in Nielsen and Chuang [32]. The baseline variant has no control structures whatsoever. This proved to be intractable, however, as even small source files can produce very large object files. One major contribution of ScaffCC is to explore techniques for mitigating this intractability.

First, the authors introduced non-flat variants of QASM — the final version being “QASM-HL”, or QASM with hierarchy and looping. These two features roughly correspond with the ideas of procedures and vectorised instructions respectively.

Next, the paper explores instrumented compilation and analysis. In practice, this is a kind of partial evaluation: the C component of the program is evaluated, and whenever a quantum call is encountered, a corresponding instruction is emitted into the compiled object file — though there is some extra bookkeeping to maintain the unflattenings in the output. The technique can be modified to produce gate count or circuit width estimates.

The paper also investigated quantum oracle generation, though it is referred to in the paper as “classical to quantum gate” (CTQG) generation. The classical gates in question must be expressed in a specific form, marking out which values are runtime and which are static.

An earlier technical report [3] contains details regarding the design of *Scaffold*, the language ScaffCC compiles. The technical report describes a much more expressive language than the Scaffold that ScaffCC compiles. It allows runtime control flow, and even measurement conditions, which are not permitted in QASM. There are also differences in how the languages handle oracle definition.

Due to the flat nature of the IR, ScaffCC’s QASM-HL is extremely limited in terms of what it can express. The compiler is also extremely permissive — it silently accepts many incorrect codes, and compiles them in unintuitive ways. For instance, consider Listing 6, which compiles down to nothing — zero instructions. The issue is that qubits cannot be allocated as individual elements — they must be part of an array.

Listing 7. Broken n -bit-rand example in OpenQASM3

```

OPENQASM 3.0;
include "stdgates.inc";
int[8] n = 10; // Should be determined by some external input
bit[10] c;
int[8] count;
bit[256] out
qubit[1] q;
while (count < n) {
  h q[0];
  c[count] = measure q[0]; // illegal; cannot index by a variable
                           // must be a compile time constant.
  count += 1;
}

```

A similar example is presented in the appendix (Listing 23); here the issue is that Scaffold does not support runtime conditions. The if statement, dependent on a bit in the circuit, compiles out to nothing.

The principal issue in both cases is that the compiler issues no warnings or errors when unsupported behaviour is used. This can be considered a serious usability issue: very often, the “obvious” reading of a program does not link up well to the actual semantics, and there is no built-in mechanism to warn users when this happens. This alone places Scaffold far away from practical usage.

The flat nature of Scaffold’s QASM is a major barrier for its adoption. Despite this, Scaffold presents many other interesting and important ideas for our consideration. In particular, its ideas of applying conventional compiler techniques have been proved successful many times.

3.2 OpenQASM

OpenQASM [9, 10] is the current de-facto standard quantum IR, in both industry and research. It was developed by IBM alongside the Qiskit quantum platform as a compiler target. OpenQASM has since seen widespread adoption and is currently on its third major version. This version is intended to function as a multi-level IR, targetting the logical and physical layers, including gate scheduling. It includes several significant additions which increase the expressiveness of the language.

OpenQASM2 programs are expressed as a list of gate applications, allocations and measurements. It does not allow any dynamic control flow — so OpenQASM2 is at **Pt. F**. In contrast, OpenQASM3 permits loops conditioned on quantum measurements — placing it at **Pt. QL**, the top of our lattice. However, OpenQASM3 lacks the ability to index registers with runtime values. All indexes must be known at compile time. Whilst this does not preclude any of our example programs, it does prevent one particularly obvious way to write the n -bit-rand example. Listing 7 demonstrates this, with the problem lines annotated with explanatory comments.

Most gates are not primitive in either version of OpenQASM. Instead, they can be derived from a small underlying set of operations. A universal rotation gate can be used to derive any 1-qubit unitary. OpenQASM2 also allows the definition of opaque gates, whose implementation is not specified. OpenQASM3 eliminates this feature. It introduces the controlled @ modifier — which produces the controlled variant of the gate its applied to. Gates can also be derived as a sequence of applications of other gates.

Some limited classical computing is allowed in the language, to support “real-time” classical co-processing — classical computing upon which the circuit continuation depends, and thus must happen within the coherence window of the qubits. The paper also discusses the notion of “near-time” (we might also call this “soft real-time”) co-processing. A typical example is data pre- or post-processing. This does not need to happen within the coherence window of the qubits. This is not supported in the language itself — instead, it must be implemented via external definitions. External definitions can also be used for real-time functionality. The actual implementation of these functions must be given in a third-party language, such as C or LLVM IR. Additionally, there is no synchronisation primitive in OpenQASM — calls to external functions do not wait for those functions to complete. This makes interleaving of classical and quantum segments difficult to represent or reason about. It is therefore not quite true to say that OpenQASM is a platform for reasoning about mixed computing. It is not possible to discuss generalised mixed programs from only within the language.

OpenQASM also supports some code re-use, through subroutines. A subroutine is essentially a function, which must declare its input and output type. These can then be used to define common operations, and factor them out of code. However, either version of OpenQASM permits only stack-based allocation. Thus, it is not possible to define custom allocators.

OpenQASM is widely used in research circles as a target for other low level components. For instance, QSSA includes mechanisms to raise OpenQASM to QSSA, and vice versa. It was designed at IBM, centring around their own physical QC implementations. It is one of the few languages which can be run on a real quantum computer. This makes OpenQASM one of the best adopted and mature low-level QPLs currently in existence.

3.3 QSSA

QSSA [36] is a static single assignment (SSA) [11] form IR for quantum computing. It is an experimental IR which explores the space for quantum IRs in the future. Among the projects goals were:

- Support for mixed computing
- Good optimisability
- Textual representation to support debugging

QSSA builds heavily on the LLVM/MLIR [30] infrastructure, manifesting as an MLIR dialect. The authors also implement an MLIR dialect representing OpenQASM, and a tool to convert conventional OpenQASM code to this dialect. The tool only works with OpenQASM2 code — which severely limits its capabilities. It also inflates the size of the file significantly. Where the OpenQASM2 implementation of Deutsch’s algorithm is only 10 lines, the OpenQASM2 MLIR translation is over 60 lines long. For this reason, we have not included QSSA samples in the appendix. QSSA can theoretically represent programs at Pt. QL — as an MLIR dialect, the existing LLVM scf (structured control flow) structures can handle control flow. However, this pipeline limits the expressiveness to that of OpenQASM2, which is at Pt. F.

One of the most important results of the QSSA project is to show that existing classical optimisation techniques transfer well to the quantum domain — even more so than was done by ScaffCC. In many cases, provided the optimisation itself does not violate logical linearity, no modification was required for it to work on QSSA code. They present an empirical analysis of their results, comparing the speed (i.e. time taken to run the optimisation) and result compactness (i.e. the gate count of the optimised result) of the transferred optimisations to existing specialist ones from Qiskit. In most cases, QSSA is shown to be superior. This is a much more tangible proof of the point than was provided initially by ScaffCC.

We found QSSA to be difficult to set up outside of the pre-built docker image provided on Zenodo. Critical parts of the infrastructure — such as the Python script which converts OpenQASM to an MLIR dialect for further work — have dependencies which are not properly recorded. QSSA does not seem to have been widely adopted. It has been at least a year since the last commit to the public repository.

3.4 QIRO

QIRO [26] is another SSA-based IR for quantum computing. It is composed of two MLIR dialects, with a lowering pass from one to the other. The first (Quantum) uses reference semantics (called “memory semantics” in the QIRO paper) for qubits, and the second (QuantumSSA) uses value semantics. The authors argue that as most existing quantum IRs use a reference semantics, adapting them to Quantum will require less effort. Thus entry to the QIRO toolchain is made easier. Quantum is intended to be the “input dialect” for the QIRO system, whereas QuantumSSA is intended to be the actual optimisation form. Much like SSA for classical computing, this makes many analyses and optimisations much easier.

The paper is principally concerned with writing optimisations for this new form, such as the loop boundary optimisation. This optimises loops with a symmetric set of quantum operations on either side by hoisting them to before and after the loop. It is among the first IRs to investigate optimising classical and quantum elements of the program simultaneously. This allows e.g. constant folding optimisations in the classical portion of the program to extend their impact to the quantum components. Investigation into more specialised quantum-classical optimisations is left for future work. It should still be noted, however, that merely the combination of classical and quantum optimisation techniques into the same infrastructure is novel. QIRO’s static optimisations are roughly as good (99.8%) as ProjectQ’s dynamic optimisations, without any of the runtime overhead.

QIRO’s programming model is fundamentally different to that of e.g. OpenQASM, as it characterises computation as fundamentally non-flat. It was designed to support co-optimisation, involving bi-directional data-flow between classical and quantum segments of the program. This places it naturally at Pt. QL. One of the fundamental assumptions laid out in the paper is that qubit allocation should be handled by the runtime environment. Qubit allocation is therefore fully dynamic in QIRO’s MLIR dialects.

Integration into a mature compiler chain makes QIRO particularly attractive. Many other quantum IRs integrate into LLVM/MLIR infrastructure (e.g. QIR, which makes its potential as an off-the-shelf optimiser or middle-end much greater. One significant hurdle to its adoption is the lack of a publicly available artifact. Prospective users must compile QIRO (and likely its dependencies, such as the specific version of LLVM) from source.

3.5 Quil

Quil [46] is an instruction set architecture (ISA) for hybrid quantum-classical computing. It is presented alongside a *quantum abstract machine* (QAM), which gives semantics to qubits and gates. This in turn is used to give semantics to Quil itself.

Quil sits at the top of the expressiveness hierarchy (Pt. QL); it includes labels (jump targets) and jump instructions, which can be conditioned on runtime values, including measurement results. It also assumes the computer has a fixed number of qubits, making allocation static.

Another key feature is the WAIT instruction, which functions as a synchronisation mechanism. The intention is that this could be used to allow interoperation with a conventional classical computer (e.g. by manipulating the classical memory of the QAM). WAIT can be used to halt the execution while the classical component is evaluated. The implementation is not specified — instead it’s intended to be implemented as required to make interoperation possible.

Listing 8. Quantum Teleportation in Quil

```

DECLARE results BIT[2]
H 0
CNOT 0 1
CNOT 3 0
H 3
MEASURE 3 results[0]
MEASURE 0 results[1]
JUMP-WHEN @THEN_0 results[0]
JUMP @END_0
LABEL @THEN_0
X 1
LABEL @END_0
JUMP-WHEN @THEN_1 results[1]
JUMP @END_1
LABEL @THEN_1
Z 1
LABEL @END_1

```

The semantics presented in the paper are not fully formal — rather, the semantics of individual elements of Quil are given as descriptions in prose. Nonetheless, this is still much more rigorous than the semantics presented for most of the other low-level quantum languages currently available. Additionally, the presentation of the QAM makes the prospect of giving a fully formal description of the semantics of Quil seem much less daunting.

Quil is far better suited to the wholesale discussions of mixed programs than e.g. OpenQASM. Its inclusion of basic classical data types such as BITS, INTEGERS and REALs and their corresponding operations make expressing rudimentary mixed programs much easier. It is not a complete solution, and there is certainly still room for improvement: For instance, REALs can be addressed by the bit range they occupy (e.g. by [0–127]), and alignment is not enforced. Bits are addressed individually, and the classical operations supported are very primitive. It is clear that Quil is not a general purpose ISA, and is highly tailored towards the quantum domain. On a practical level, this is somewhat mitigated by the inclusion of the WAIT feature. Quil is still certainly much better suited to mixed computing than many of its competitors.

We should also acknowledge the Quil-T annex [1], which adds support for gate timings to the language. Quil seems to have been the pioneer in this area, with the OpenQASM support following its example.

Quil was proposed by Rigetti computing, a quantum computing startup focused on superconducting quantum computers. Rigetti already support Quil natively on their quantum products, and also provide the Pyquil Python library³. Pyquil provides facilities for compiling Quil programs, simulating them on a virtual machine, or using quantum services available in the cloud.

3.6 Summary

Table 1 provides a summary of our findings. Quantum IRs have seen an explosion of activity since the ideas were first explored by Scaffold. Through such projects, we have gained insight into the difficulties of working with flat codes (Scaffold) and how we can use multi-layered compiler

³<https://github.com/rigetti/pyquil?tab=readme-ov-file>

Language	CCF Lattice Point	Qubit Allocation	Execution Platform(s)
Scaffold [27]	F (Flat)	Static	-
OpenQASM v2 [10]	CC	Stack-based	IBM QE
OpenQASM v3 [9]	QL	Stack-based	IBM QE
Quil [46]	QL	Static	Rigetti QCS
QSSA [36]	QL	Stack-based	IBM QE ⁴
QIRO [26]	QL	Dynamic	-

Table 1. Summary of low-level QPLs and their features

architectures to tackle problems like logical optimisation before considering how we need to adapt the program to a specific qubit architecture (Quil). Much still remains to do in this space, however. We do not believe that any of the IRs presented in this survey constitute the final word on the topic.

A large body of work now exists showing the effectiveness of classical compiler techniques, when translated to the quantum domain. Despite this, in practice such techniques are not leveraged to great effect. Most quantum IRs (certainly the influential ones) use compilers written from scratch with custom optimisation passes.

It is particularly interesting to compare QIRO and QSSA. Both are MLIR dialects which use SSA and value semantics in optimising quantum programs. The projects have similar aims, similar approaches, but reach different conclusions. QSSA's key finding is that modification of the underlying MLIR passes is not necessary to achieve excellent results. QIRO, on the other hand, does implement its own custom optimisations, and demonstrates how we can adapt the infrastructure to the mixed-quantum domain. There is certainly scope for an in-depth comparative study of the two, though we will not address such a topic here.

Much like in classical computing, there is a lack of semantic grounding for low-level quantum languages. Quil is the best example in this space, with at least some formalisms for some of its more fundamental operations, but it lacks a full formal treatment.

On a technical basis, many existing quantum IRs are fully expressive. In practice, however, things are much more complicated. While it is theoretically possible to write arbitrary mixed programs in OpenQASM, doing so is laborious, and requires foreign function calls. Quil is better, with primitive integers and associated operations, but the classical elements are secondary and immature. QSSA and QIRO, whilst supporting mixed computing, are both fundamentally forms for optimisation, and thus are constrained by whatever languages they will eventually be lowered to. This is a substantial hole in the landscape.

4 High-Level Approaches

High-level approaches in programming languages are generally focused on making it easier for the human to write programs. This can be achieved in many ways — for instance, it may be that we are looking to provide certain guarantees about programs. On the other hand, it may be that we wish to allow the programmer to express the same thing in a simpler way — or to simply require less work to do the same thing.

QPLs present plenty of problems which would benefit from high-level approaches. In §2.2.3 we discussed features arising from reversibility (i.e. inverse generation or oracle generation). These features must both be dealt with at a high level of abstraction.

⁴Via lowering to OpenQASM

Quantum memory management is another concern which is generally dealt with at a high level. Approaches to this vary. We can enforce logical linearity e.g. with a type system. A well-typed program in such a type theory will manage its qubits properly, but the actual management is entirely manual — it must be handled by the programmer. On the other end of the spectrum, we may wish to fully automate the management of qubits. There is a range of solutions in between, allowing for some amount of automation.

It's rare to see a language which fully tackles all of these problems. Many of the languages we will discuss here are research prototypes, designed to explore how to tackle just one. We shouldn't expect these languages to be mature; that is not their aim.

Still, it's useful to consider how far from production they are. Thus, for each language, we will examine which platforms it supports as compile targets. We will also touch on expressiveness (in the framing of the CCF lattice) and general language ergonomics, meaning which facilities are available to the programmer: functions, automated memory management, automated validity checks, etc.

4.1 Python Libraries

We would be remiss to leave out the wide array of Python libraries designed for quantum computing. These are part of a larger family of quantum embedded domain specific languages (eDSLs). Examples of tools in this class are Qiskit⁵ [28] Pyquil⁶, Cirq⁷ and ProjectQ [49]. Samples for each of these languages can be found in §C.

These libraries are arguably the easiest way to approach quantum computing. In many cases, they can simply be “pip install”-ed with no other requirements. They often ship with a built-in simulator that makes it easy to begin playing around with the basic concepts of quantum computing and quantum algorithms. This makes these libraries particularly well suited to teaching.

Qiskit is the most popular of these languages [2], and also typical in many regards. Qiskit is technically very capable — it compiles down to OpenQASM3, which sits at Pt. QL. Qiskit maintains this level of control. Despite this, Qiskit's facilities for mixed computing are very poor. It inherits OpenQASM's inability to represent the n -bit-rand example. However, it is also incapable of nicely representing the geometric sampler. There is no notion of circuit-runtime integers in Qiskit, so any operations on classical integers would need to be implemented by hand on bits. Additionally, Qiskit also does not provide any mechanism for manipulating bits directly. This leaves only implementing an adder on qubits, and measuring the result out onto classical bits. We argue however, that this does not truly solve the problem. Operations such as classical integer addition are much better suited for execution on a classical computer. Thus QPLs which aim to tackle the mixed space need to provide good facilities for classical computing at their runtime. These two problem examples also do not condition circuit continuations on these computations; it is entirely post-circuit processing. This is the kind of mixed computing that current quantum computers are capable of.

Whilst libraries like Qiskit are very approachable for a human, their embedding into Python becomes a major weakness. Python is poorly suited to static analysis, meaning that many approaches to solving common QPL problems are simply not feasible. ProjectQ is interesting in this regard as it is a full compiler chain implemented in python. The benefit of a fresh toolchain is that many new ideas can be explored. The ProjectQ compiler is flexible — new modules representing different optimisation strategies can be added by simply including them in the list of stages in the compiler engine. It is also lazy, in that it generates instructions only on demands. A compiler stage holds

⁵<https://www.ibm.com/quantum/qiskit>

⁶<https://github.com/rigetti/pyquil>

⁷<https://quantumai.google/cirq>

only a limited number of instructions in memory at a time. This *locality of optimisation* makes ProjectQ capable of handling much larger programs.

Adoption of these libraries varies a lot. For instance, Qiskit is very popular in education (Glasgow’s own QC Course is taught using Qiskit) and in research, because of its accessibility, simplicity and ability to compile to OpenQASM (which makes it suitable for lots of tasks via export). Cirq and Pyquil are both less popular, perhaps because of expressiveness issues (Cirq is at [Pt. F](#)) or the lack of popularity of the underlying IR.

4.2 Quipper

Quipper [20, 21] is another Quantum eDSL, this time embedded into Haskell. Similar to its Python counterparts, it requires only a “cabal install”. It has been very influential, being the originator of the two stage compiler pipeline, which many others would later build on.

Quipper conceptualises program execution to happen in three phases: (1) compilation (of the underlying Haskell), (2) circuit generation, and (3) circuit execution. Quipper (particularly in Green et al. [20]) is very careful to disambiguate which data exists at each phase of execution.

Parameters are classical values which are known at the time of circuit generation. These are generally Haskell-native data-types such as Bool. In contrast, *Bits* are classical boolean values which are only known at circuit execution time — in general, these are called *states* or *inputs*. Of course, Bits can also be created via the measurement of Qubits, the other circuit-execution time data type.

It is possible convert a Bit to a Bool, through the use of the `dynamic_lift` function. This is necessary to facilitate a bi-directional data-flow for mixed computing, and thus reach higher points in the CCF lattice. With the inclusion of dynamic lifting, we reach [Pt. QL](#).

Dynamic lifting is not needed for all programs at [Pt. QC](#). For instance, it is not necessary for quantum teleportation (as shown in Listing 9).

Listing 9. Quantum teleportation in Quipper

```
import Quipper

bellPair :: Circ (Qubit, Qubit)
bellPair = do
  a <- qinit false >=> hadamard
  b <- qinit false
  b <- gate_X b `controlled` a
  return (a, b)

teleport :: Qubit -> Circ Qubit
teleport q = do
  (a, b) <- bellPair
  q <- gate_X q `controlled` a >=> hadamard >=> measure
  a <- measure a
  b <- gate_X b `controlled` a
  b <- gate_Z b `controlled` q
  cdiscard (a, q)
  return b

main :: IO ()
main = print_generic PDF $ teleport (qinit False)
```

However, issues arise when we want to include more behaviour. If we wish to compute arbitrary classical functions, dependent on measurement results, we require dynamic lifting.

Loops also require dynamic lifting, as demonstrated by the n -bit-rand example. The n -bit-rand algorithm generates an n -bit random number. The key idea is that n is known only at runtime. In Quipper, this begs the question of “which runtime?” If it is to be known at circuit generation time, there is no issue. In fact, this admits one of the simplest implementations of the n -bit-rand algorithm (Listing 10).

Listing 10. n -bit-rand in Quipper

```
import Quipper

nBitRand :: Int -> Circ [Bit]
nBitRand 0 = []
nBitRand count = do
  newBit <- qinit False >>= hadamard >>= measure
  otherBits <- nBitRand $ count - 1
  return newBit : otherBits
```

However, circuit generation time is analogous to the compile time of languages like Scaffold. The principal point is that the data should only be known at *circuit runtime*. This becomes much more troublesome to implement. Quipper does not have any looping constructs in its circuit runtime, and so instead we must escape to Haskell in order to perform the recursion. Furthermore, quipper lacks other primitive data-types at circuit-execution time. Thus, much like in the Python libraries we have already seen, we are reduced to manual bit manipulation to accomplish even relatively simple tasks such as decrementing an integer or checking if it is zero. This results in a much messier implementation, as in Listing 11.

Listing 11. n -bit-rand in Quipper with dynamic lifting

```
import Quipper

type CInt = [Bit]

cIntZero :: CInt -> Circ Bit
cIntZero [] = cinit True
cIntZero (x : xs) =
  if x == cinit True
  then cinit False
  else cIntZero xs

cIntDec :: CInt -> Circ CInt
cIntDec [] = []
cIntDec (bit : rest) = do
  restIsZero <- cIntZero rest
  bit <- cnot bit `controlled` restIsZero
  rest <- cIntDec rest
  return (bit : rest)
```

```

nBitRand :: [Bit] -> Circ [Bit]
nBitRand count = do
  isZero <- cIntZero rest >=> dynamic_lift

  if isZero
  then return []
  else do
    count <- cIntDec count
    newBit <- qinit False >=> hadamard >=> measure
    otherBits <- nBitRand count
    return newBit : otherBits

```

The necessity of dynamic lifting in these looping programs also confuses the clean distinctions between execution phases. If we freely move between generation and execution to achieve some computational end, is the distinction between phases meaningful?

A key takeaway of this is that Quipper does not present a fundamentally new programming model. Though it is more ergonomic to use than many of its contemporaries, it is still bound by the limitations of the underlying model — that of the flat circuit. It is not suitable for discussing mixed programs.

Rather, the main contribution of the Quipper project has been as a basis for semantics research. Fragments of Quipper have been extracted (referred to as “proto-quipers”) and studied. The goal is to give an interesting semantics to the fragment, which provides a mathematical basis for reasoning about some feature of Quipper. At present, fragments include:

- **proto-quipper-dyn** [14] which explores the semantics of dynamic lifting
- **proto-quipper-D** [13] with linear dependent types
- **proto-quipper-M** [16, 43], also with linear dependent types
- **proto-quipper-S** [44] with a type system that guarantees reversible circuits
- **proto-quipper-C** [15] with semantics for gate modifiers (reversing and control).

The long term goal of the project is to unify the fragments and gain a well defined semantics for the whole programming language. This is a mammoth task, which has proved thus far elusive.

Quipper uses a value semantics for its qubit names. It is embedded into Haskell, and therefore inherits its type system, and is unable to enforce logical linearity on qubits. However, some partial linearity can be guaranteed through the use of the shadowing convention. By always binding the result of a gate application to the same names on which it operated, the old values are *shadowed* and become inaccessible. If this convention is always followed, then qubits will be used affinely, and the program will at least be physical. However, affinity is not sufficient to guarantee correctness; qubits must be used *exactly once* to avoid problems related to implicit measurement.

Natively, Quipper does not compile to a quantum assembly. This makes actually executing Quipper programs very difficult. Quipper ships with a custom simulator. The support for simulating dynamic lifting is experimental, and the documentation is poor. Unfortunately, we could not get this working. This alone places Quipper a long way from being production ready. Installing the Quipper toolchain can be somewhat difficult, requiring specific versions of GHC. We could not get it working on the Author’s Macbook Air. Some preliminary work by Wesley [55] explores transpiling Quipper source code to OpenQASM, and vice-versa. There is also an ongoing re-write of Quipper⁸, focused on matching the features of proto-quipper.

⁸<https://gitlab.com/frank-peng-fu/dpq-remake>

4.3 Silq

Silq [6] is a stand-alone high-level language for quantum computing. Its principle contribution is a novel type system, which can check if scratch values can be automatically uncomputed. This is a significant contribution for two reasons. Firstly, such scratch values are very common. Grover’s algorithm for instance relies on this kind of scratch value — an oracle function computes a condition, which is used to conditionally apply a phase change. Secondly, handling this kind of work manually is tedious and prone to error. It is exactly the sort of task which benefits from automation.

It’s important to note that Silq itself does not implement the uncomputation. The type system merely guarantees that temporary values can be uncomputed. Actual uncomputations must be generated by an auxiliary system, such as Unqomp [34].

When Silq cannot work out an uncomputation strategy for a qubit, it enforces that the qubit should be used exactly once (logical linearity). Thus, it rejects the sample we gave earlier, which breaks stack based allocation (Listing 3). A translation of this program into Silq is given in Listing 57.

The semantics of Silq are fully expressive, placing it at Pt. QL. However, Silq does not have a compiler, and cannot lower to a quantum assembly. This is a significant obstacle to its adoption for use in real quantum computing applications. The recommended VSCode plugin can perform some simulation, but it is a long way from production ready.

Whilst Silq’s type system is state-of-the-art, improvements may be possible. For instance Silq requires that a given scratch bit undergoes only “qfree” operations — operations which do not induce any superposition in the qubit. However, this is a pessimistic analysis; there are cases in which superposition states can be uncomputed.

A more recent example of work in this vein is given by Hirata and Heunen [25]. Their language, Qurts, provides an implementable semantics for and discards the “qfree” requirement.

4.4 Q#

Q# [50] is a standalone DSL for quantum computing developed at Microsoft. Unlike many of its peers, it presents a fundamentally different programming model. Q# conceptualises quantum computers as coprocessors, and moves away from the circuit runtime model. It allows non-flat program structures at a primitive level, and also supports a bi-directional data flow between classical and quantum program elements. This places Q# at Pt. QL.

Qubit usage in Q# is strictly stack-based. Qubits are managed via a `using` statement. This allocates a collection of qubits for the duration of an associated block of code. The qubits are then deallocated at the end of this block. This is essentially equivalent to a stack-based approach — although it is decoupled from function calls and thus is more granular than a typical stack implementation. Q# also offers an equivalent statement (borrowing) that enables the use of *dirty ancillae*; no other language thus far discussed facilitates this in a first class manner.

Code reuse is enabled by *operations*. Operations are segments of code which *may* have quantum side effects. They are differentiated from *functions*, which must be pure classical computations. It is also possible to specify variants of operations, for instance the adjoint (inverse) operation, or a controlled variant. In some cases, Q# is able to generate these variants automatically.

Mixed computing support is limited. Q# is not a general-purpose programming language — it is very much targetted at the quantum domain. Consider strings as an example. Though Q# has a string data type, they exist only for reporting information back to a higher-level scheduler. Manipulation of strings is extremely limited, allowed only through templates.

Q# seem to be a fairly mature language, when compared to its peers. It explores many cutting edge ideas which are difficult to find elsewhere (e.g. dirty ancillae, automatic generation) — certainly in combination. QIR, the IR underlying the Q# compilation pipeline, is backed by a consortium and

Language	CCF Lattice Point	Qubit Allocation	Compile Targets
Qiskit [28]	QL	Static	OpenQASM
Cirq ⁹	Flat	Static	OpenQASM
Pyquil ¹⁰	QL	Static	Quil
Quipper [21]	QL	Dynamic	OpenQASM ¹¹
Q# [50]	QL	Stack-based	QIR
Silq [6]	QL	Dynamic	-

Table 2. Summary of high-level QPLs and their features

is, to some extent, an industry standard. The Q# compiler is well established, and the toolchain is frequently discussed in the literature.

4.5 Summary

Languages like Qiskit are more ergonomic than raw OpenQASM. However, the programming model they represent is not fundamentally different from OpenQASM. It's a low-level circuit manipulation framework.

In contrast, Silq has many of the features we might like to see in a real high-level QPL. But Silq does not have a compiler toolchain. Thus it's also unsuitable for real production use.

At present, a production-ready high-level quantum programming language does not exist. We still lack a stand-alone high-level quantum language which is capable of compiling down to a quantum assembly, which has the kinds of ergonomics that make writing software at scale possible. However, Quipper's new OpenQASM compiler [55] is a step in the right direction.

5 Related Work

For related work, we first turn to other surveys in the space. Gay's 2006 survey [17] provides good coverage of a similar area, though it is now somewhat dated. Sofge [48] provides a small update to this a few years later. Cardama et al. [7] provide a more recent review, covering quantum IRs; it has different goals to this paper and also accounts for IRs targetting distributed quantum computation. Heim et al. [23] provide a very modern and generalised view of quantum computing. For a thorough background in quantum computing, we refer to Nielsen and Chuang [32] and Coecke and Kissinger [8].

We regret that there was not scope to make this a truly comprehensive survey of the space. Low-level approaches we did not cover in-depth are QIR and XACC [31]. There are also several QIRs which deal with networked quantum computing; for an analysis of these we direct readers to the latter half of Cardama et al. [7]. High-level languages we did not discuss include QWIRE (and ReQWIRE) [35, 42], Liqui |> [54], Qurts [25], Quff [57], Twist [59], Qunity [52] and Qwerty/ASDF [4].

6 Conclusions

Our most significant contribution is the CCF lattice. We believe this framework provides a novel and valuable way to analyse the capabilities of quantum programming languages and hardware architectures, as well as a way to classify algorithms. A valuable piece of potential future work would be to develop a semantic formalism for the framework, to enable the study of the properties of the different points.

¹¹<https://quantumai.google/cirq>

¹¹<https://github.com/rigetti/pyquil>

¹¹Due to Wesley[55]

In terms of findings in existing languages; it is clear that low-level quantum languages are not yet ready to describe mixed programs. Though some of the newer optimising forms (such as QSSA or QIRO) are capable of expressing more complex control flow, they depend on older QIRs as output dialects, which do not provide such support. It's also clear that semantics research for low-level languages is severely lacking, with only Quil providing a *partial* semantics.

It's apparent that techniques developed in classical language research translate well to the quantum domain. This is true in optimisation (as shown in Scaffold and QSSA), but also in higher level structures, as demonstrated by Silq's type system.

High level approaches have been used to explore and solve many problems unique to the quantum domain, from inverse and oracle generation, to qubit management. Despite this, high-level quantum languages are immature, in the sense that they are far from practical use. Some of this is inherited from the immaturity of low-level languages, but this cannot solely be blamed. Silq, perhaps the most ergonomic of the languages surveyed here, lacks a compiler entirely. And whilst in the case of Silq, the aims of the project were more towards design than implementation, the fact remains that a robust compiler toolchain for quantum compilation simply does not exist.

Most high-level languages *with* a compiler fundamentally tie themselves into the circuit model. This model is inflexible and lacks expressiveness, in terms of CCF. A substantial piece of future work will be to develop a compiler toolchain which can support high-level languages with these higher CCF expressiveness capabilities. This will also require a more nuanced runtime model than simple circuits.

Some of the most valuable work remaining is applying well understood techniques from classical compilers to build up a mature infrastructure ecosystem, which can support the more exciting ideas being developed in quantum language design.

References

- [1] [n. d.]. Quil-T. <https://quil-lang.github.io/#12Annex-T--Pulse-Level-Control>.
- [2] 2025. 2025 Quantum Open Source Survey. <https://unitaryfoundation.github.io/survey-2025/>.
- [3] Ali Javadi Abhari, Arvin Faruque, Mohammad Javad Dousti, Lukas Svec, Oana Catu, Amlan Chakrabati, Chen-Fu Chiang, Seth Vanderwilt, John Black, Fred Chong, et al. 2012. Scaffold: Quantum programming language. *Princeton Univ NJ Dept of Computer Science* (2012). <https://www.cs.princeton.edu/techreports/2012/934.pdf>
- [4] Austin J. Adams, Sharjeel Khan, Arjun S. Bhamra, Ryan R. Abusaada, Anthony M. Cabrera, Cameron C. Hoechst, Travis S. Humble, Jeffrey S. Young, and Thomas M. Conte. 2025. ASDf: A Compiler for Qwerty, a Basis-Oriented Quantum Programming Language. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization* (Las Vegas, NV, USA) (CGO '25). Association for Computing Machinery, New York, NY, USA, 444–458. doi:10.1145/3696443.3708966
- [5] Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. 1993. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Phys. Rev. Lett.* 70 (Mar 1993), 1895–1899. Issue 13. doi:10.1103/PhysRevLett.70.1895
- [6] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: a high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 286–300. doi:10.1145/3385412.3386007
- [7] F. Javier Cardama, Jorge Vázquez-Pérez, César Piñeiro, Juan C. Pichel, Tomás F. Pena, and Andrés Gómez. 2025. Review of intermediate representations for quantum computing. *The Journal of Supercomputing* 81, 2 (21 Jan 2025), 418. doi:10.1007/s11227-024-06892-2
- [8] Bob Coecke and Aleks Kissinger. 2017. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press.
- [9] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. 2022. OpenQASM 3: A Broader and Deeper Quantum Assembly Language. *ACM Transactions on Quantum Computing* 3, 3, Article 12 (Sept. 2022), 50 pages. doi:10.1145/3505636

- [10] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. 2017. Open Quantum Assembly Language. (2017). arXiv:1707.03429 [quant-ph] <https://arxiv.org/abs/1707.03429>
- [11] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. 1991. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Program. Lang. Syst.* 13, 4 (Oct. 1991), 451–490. doi:10.1145/115372.115320
- [12] David Deutsch and Richard Jozsa. 1992. Rapid solution of problems by quantum computation. *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences* 439, 1907 (12 1992), 553–558. doi:10.1098/rspa.1992.0167 arXiv:<https://royalsocietypublishing.org/rspa/article-pdf/439/1907/553/68698/rspa.1992.0167.pdf>
- [13] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2020. A Tutorial Introduction to Quantum Circuit Programming in Dependently Typed Proto-Quipper. In *Reversible Computation (RC '20) (Lecture Notes in Computer Science, Vol. 12227)*, Ivan Lanese and Mariusz Rawski (Eds.). Springer International Publishing, Cham, 153–168. doi:10.1007/978-3-030-52482-1_9 arXiv:2005.08396
- [14] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 11 (January 2023), 11 pages. doi:10.1145/3571204 arXiv:2204.13041
- [15] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2025. Proto-Quipper with Reversing and Control. *Electronic Proceedings in Theoretical Computer Science* 426 (Aug. 2025), 1–22. doi:10.4204/eptcs.426.1
- [16] Peng Fu, Kohei Kishida, and Peter Selinger. 2022. Linear Dependent Type Theory for Quantum Programming Languages. *Logical Methods in Computer Science* 18, 3, Article 28 (September 2022), 28 pages. doi:10.46298/lmcs-18(3:28)2022
- [17] Simon J. Gay. 2006. Quantum programming languages: survey and bibliography. *Mathematical Structures in Computer Science* 16, 4 (2006), 581–600. doi:10.1017/S0960129506005378
- [18] Jean-Yves Girard. 1987. Linear logic. *Theoretical Computer Science* 50, 1 (1987), 1–101. doi:10.1016/0304-3975(87)90045-4
- [19] Jean-Yves Girard. 1995. Linear logic: its syntax and semantics. In *Proceedings of the Workshop on Advances in Linear Logic*. Cambridge University Press, USA, 1–42.
- [20] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. An Introduction to Quantum Programming in Quipper. In *Reversible Computation*, Gerhard W. Dueck and D. Michael Miller (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 110–124.
- [21] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *CoRR* abs/1304.3390 (2013). arXiv:1304.3390 <http://arxiv.org/abs/1304.3390>
- [22] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (Philadelphia, Pennsylvania, USA) (STOC '96). Association for Computing Machinery, New York, NY, USA, 212–219. doi:10.1145/237814.237866
- [23] Bettina Heim, Mathias Soeken, Sarah Marshall, Chris Granade, Martin Roetteler, Alan Geller, Matthias Troyer, and Krysta Svore. 2020. Quantum programming languages. *Nature Reviews Physics* 2, 12 (01 Dec 2020), 709–722. doi:10.1038/s42254-020-00245-7
- [24] Jessie M. Henderson, Elena R. Henderson, Aviraj Sinha, Mitchell A. Thornton, and D. Michael Miller. 2023. Automated quantum oracle synthesis with a minimal number of qubits. In *Quantum Information Science, Sensing, and Computation XV*, Eric Donkor, Michael Hayduk, Michael L. Fanto, and Carlos M. Torres Jr. (Eds.), Vol. 12517. International Society for Optics and Photonics, SPIE, 1251706. doi:10.1117/12.2663240
- [25] Kengo Hirata and Chris Heunen. 2024. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. arXiv:2411.10835 [cs.PL] <https://arxiv.org/abs/2411.10835>
- [26] David Ittah, Thomas Häner, Vadym Kliuchnikov, and Torsten Hoefler. 2022. QIRO: A Static Single Assignment-based Quantum Program Representation for Optimization. *ACM Transactions on Quantum Computing* 3, 3, Article 14 (June 2022), 32 pages. doi:10.1145/3491247
- [27] Ali Javadi-Abhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. 2014. Scaffold: a framework for compilation and analysis of quantum computing programs. In *Proceedings of the 11th ACM Conference on Computing Frontiers* (Cagliari, Italy) (CF '14). Association for Computing Machinery, New York, NY, USA, Article 1, 10 pages. doi:10.1145/2597917.2597939
- [28] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph] <https://arxiv.org/abs/2405.08810>
- [29] E. Knill. 1996. Conventions for Quantum Pseudocode. arXiv:2211.02559 [quant-ph] <https://arxiv.org/abs/2211.02559>
- [30] Chris Lattner, Jacques A. Pienaar, Mehdi Amini, Uday Bondhugula, River Riddle, Albert Cohen, Tatiana Shpeisman, Andy Davis, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A Compiler Infrastructure for the End of Moore's Law. *CoRR* abs/2002.11054 (2020). arXiv:2002.11054 <https://arxiv.org/abs/2002.11054>
- [31] Alexander J. McCaskey, Dmitry I. Lyakh, Eugene F. Dumitrescu, Sarah S. Powers, and Travis S. Humble. 2019. XACC: A System-Level Software Infrastructure for Heterogeneous Quantum-Classical Computing. arXiv:1911.02452 [quant-ph] <https://arxiv.org/abs/1911.02452>

- [32] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press.
- [33] M. Oskin, F.T. Chong, and I.L. Chuang. 2002. A practical architecture for reliable quantum computers. *Computer* 35, 1 (2002), 79–87. doi:10.1109/2.976922
- [34] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: synthesizing uncomputation in Quantum circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 222–236. doi:10.1145/3453483.3454040
- [35] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: a core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (Paris, France) (POPL '17)*. Association for Computing Machinery, New York, NY, USA, 846–858. doi:10.1145/3009837.3009894
- [36] Anurudh Peduri, Siddharth Bhat, and Tobias Grosser. 2022. QSSA: an SSA-based IR for Quantum computing. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction (Seoul, South Korea) (CC 2022)*. Association for Computing Machinery, New York, NY, USA, 2–14. doi:10.1145/3497776.3517772
- [37] Simon Perdrix. 2007. Quantum Patterns and Types for Entanglement and Separability. *Electronic Notes in Theoretical Computer Science* 170 (2007), 125–138. doi:10.1016/j.entcs.2006.12.015 Proceedings of the 3rd International Workshop on Quantum Programming Languages (QPL 2005).
- [38] Simon Perdrix. 2008. Quantum Entanglement Analysis Based on Abstract Interpretation. In *Proceedings of the 15th International Symposium on Static Analysis (Valencia, Spain) (SAS '08)*. Springer-Verlag, Berlin, Heidelberg, 270–282. doi:10.1007/978-3-540-69166-2_18
- [39] Simon Perdrix and Philippe Jorrand. 2006. Classically-controlled Quantum Computation. *Electron. Notes Theor. Comput. Sci.* 135, 3 (March 2006), 119–128. doi:10.1016/j.entcs.2005.09.026
- [40] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (Aug. 2018), 79. doi:10.22331/q-2018-08-06-79
- [41] Frédéric Prost and Chaouki Zerrari. 2009. *Reasoning about Entanglement and Separability in Quantum Higher-Order Functions*. Springer Berlin Heidelberg, 219–235. doi:10.1007/978-3-642-03745-0_25
- [42] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (Jan. 2019), 299–312. doi:10.4204/eptcs.287.17
- [43] Francisco Rios and Peter Selinger. 2018. A Categorical Model for a Quantum Circuit Description Language (Extended Abstract). In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL), Nijmegen, the Netherlands, July 3–7, 2017 (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*, Bob Coecke and Aleks Kissinger (Eds.). Open Publishing Association, Waterloo, NSW, Australia, 164–178. doi:10.4204/EPTCS.266.11
- [44] Neil J. Ross. 2017. Algebraic and Logical Methods in Quantum Computation. arXiv:1510.02198 [quant-ph] <https://arxiv.org/abs/1510.02198>
- [45] Peter Selinger. 2004. Towards a quantum programming language. *Mathematical Structures in Comp. Sci.* 14, 4 (Aug. 2004), 527–586. doi:10.1017/S0960129504004256
- [46] Robert S. Smith, Michael J. Curtis, and William J. Zeng. 2016. A Practical Quantum Instruction Set Architecture. *CoRR* abs/1608.03355 (2016). arXiv:1608.03355 <http://arxiv.org/abs/1608.03355>
- [47] Mathias Soeken and Mariia Mykhailova. 2022. Automatic oracle generation in microsoft’s quantum development kit using QIR and LLVM passes. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (San Francisco, California) (DAC '22)*. Association for Computing Machinery, New York, NY, USA, 1363–1366. doi:10.1145/3489517.3530626
- [48] Donald A. Sofge. 2008. A Survey of Quantum Programming Languages: History, Methods, and Tools. In *Proceedings of the Second International Conference on Quantum, Nano and Micro Technologies (ICQNM 2008) (ICQNM '08)*. IEEE Computer Society, USA, 66–71. doi:10.1109/ICQNM.2008.15
- [49] Damian S. Steiger, Thomas Häner, and Matthias Troyer. 2018. ProjectQ: an open source software framework for quantum computing. *Quantum* 2 (Jan. 2018), 49. doi:10.22331/q-2018-01-31-49
- [50] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (Vienna, Austria) (RWDSDL2018)*. Association for Computing Machinery, New York, NY, USA, Article 7, 10 pages. doi:10.1145/3183895.3183901
- [51] Mads Tofte and Jean-Pierre Talpin. 1997. Region-Based Memory Management. *Information and Computation* 132, 2 (1997), 109–176. doi:10.1006/inco.1996.2613
- [52] Finn Voichick, Liyi Li, Robert Rand, and Michael Hicks. 2023. Qunity: A Unified Language for Quantum and Classical Computing. *Proc. ACM Program. Lang.* 7, POPL, Article 32 (Jan. 2023), 31 pages. doi:10.1145/3571225

- [53] Philip Wadler. 1993. Linear Types Can Change the World! (10 1993).
- [54] Dave Wecker and Krysta M. Svore. 2014. LIQ*Ui* | $\rangle$: A Software Design Architecture and Domain-Specific Language for Quantum Computing. arXiv:1402.4467 [quant-ph] <https://arxiv.org/abs/1402.4467>
- [55] Scott Wesley. 2024. LinguaQuanta: Towards a Quantum Transpiler Between OpenQASM and Quipper. In *Reversible Computation - 16th International Conference, RC 2024, Toruń, Poland, July 4-5, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 14680)*, Torben Ægidius Mogensen and Lukasz Mikulski (Eds.). Springer, 142–160. doi:10.1007/978-3-031-62076-8_10
- [56] W. K. Wootters and W. H. Zurek. 1982. A single quantum cannot be cloned. *Nature* 299, 5886 (01 Oct 1982), 802–803. doi:10.1038/299802a0
- [57] Christopher John Wright, Mikel Luján, Pavlos Petoumenos, and John Goodacre. 2024. Quff: A Dynamically Typed Hybrid Quantum-Classical Programming Language. In *Proceedings of the 21st ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (Vienna, Austria) (MPLR 2024)*. Association for Computing Machinery, New York, NY, USA, 65–81. doi:10.1145/3679007.3685063
- [58] Shangzhou Xia and Jianjun Zhao. 2023. Static Entanglement Analysis of Quantum Programs. In *2023 IEEE/ACM 4th International Workshop on Quantum Software Engineering (Q-SE)*. 42–49. doi:10.1109/Q-SE59154.2023.00013
- [59] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: sound reasoning for purity and entanglement in Quantum programs. *Proc. ACM Program. Lang.* 6, POPL, Article 30 (Jan. 2022), 32 pages. doi:10.1145/3498691

A Quantum Fundamentals

A.1 Complex Numbers

A complex number $x \in \mathbb{C}$ is described by two real numbers: a real part and an imaginary part. The *conjugate* (written $\bar{x}$) of a complex number inverts the sign of the imaginary part. All real numbers are their own complex conjugate.

$$x = a + bi \quad \bar{x} = a - bi$$

The *modulus* of a complex number $|x|$ is given by adding the sum of the squares of its coefficients, vis.

$$x = a + bi \quad |x| = \sqrt{a^2 + b^2}$$

A.2 Matrices and Vectors

For a complex-valued matrix A , its *conjugate transpose* A^* is given by taking the conjugate for every entry and transposing it. For example:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^* = \begin{bmatrix} \bar{a} & \bar{c} \\ \bar{b} & \bar{d} \end{bmatrix}$$

The A^* notation is common in linear algebra. A^H is also frequently used there. $A^\dagger$ is particularly common in quantum mechanics and QPL research.

The *kroncker product* of two vectors can be defined:

$$\begin{bmatrix} a \\ b \\ \vdots \\ z \end{bmatrix} \otimes v_2 = \begin{bmatrix} a \cdot v_2 \\ b \cdot v_2 \\ \vdots \\ z \cdot v_2 \end{bmatrix}$$

Wherein $\begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$ represents the concatenation of v_1 and v_2 . This is really just a special case of the more general matrix kronecker product, defined below:

$$\begin{bmatrix} a_{0,0} & \dots & a_{0,j} \\ \vdots & \ddots & \vdots \\ a_{i,0} & \dots & a_{i,j} \end{bmatrix} \otimes B = \begin{bmatrix} a_{0,0}B & \dots & a_{0,j}B \\ \vdots & \ddots & \vdots \\ a_{i,0}B & \dots & a_{i,j}B \end{bmatrix}$$

Again, here $\begin{bmatrix} A & B \\ C & D \end{bmatrix}$ represents the concatenation of A , B , C and D (called the blocks or submatrices).

A.3 Qubits

The fundamental unit of quantum information is called a *qubit* (quantum bit). A qubit may be in one of two *basis states* — either $|0\rangle$ or $|1\rangle$ — or it may be in a more complex *superposition state*. Generally, a qubit $|\psi\rangle$ is described as

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad \text{for some } \alpha, \beta \in \mathbb{C} \\ |\alpha|^2 + |\beta|^2 = 1$$

where $\mathbb{C}$ is the complex numbers. The coefficients α and β (called the amplitudes) relate to the probability of measuring the qubit as either 0 or 1, respectively:

$$P(0) = |\alpha|^2 \quad P(1) = |\beta|^2$$

Measurement is destructive: after measuring a qubit, it will “collapse” into one of the basis states. Multiple qubit states are described in a similar way:

$$\begin{aligned} |\psi\rangle &= \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle \\ &\text{for some } \alpha, \beta, \gamma, \delta \in \mathbb{C} \\ |\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 &= 1 \end{aligned}$$

where again the coefficients relate to the probability of measuring the qubits in the corresponding (composite) basis state. This notation (known as Dirac or “bra-ket” notation) is in reality just a way of working with vectors. A qubit is a vector:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

and $|0\rangle$ and $|1\rangle$ are just special cases:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

For multi-qubit states, the rows count up in binary:

$$|\phi\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle = \begin{bmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{bmatrix}$$

Some states can be described as a tensor product between two sub-states. For example:

$$\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

In this case, we say that the state is *separable* into its two sub-components. But states also exist which cannot be described in such a manner. The Bell states on two qubits are a set of four two-bit states, in which the pair either always agree, or always disagree. $|\phi^+\rangle$ in which both qubits always agree, is shown below:

$$|\phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

We call these non-separable states *entangled*. It is not possible to fully describe entangled states compositionally. Measuring one qubit in an entangled pair collapses not only that qubits’ state, but also that of its entangled partner. For example, consider the $|\phi^+\rangle$ state from above. When measured, both will be in the same basis state. In fact, it is stronger than this; the second qubit will collapse into the basis state after the first is measured. Einstein called this “spooky action at a distance”, and it is an example of non-local behaviour. This sort of behaviour is at the heart of many quantum key distribution algorithms.

When presenting algorithms, the convention is to work with *logical qubits* unless otherwise stated. Logical qubits are perfect — they experience no noise, and admit no errors. The idea is to allow a discussion of the algorithm in its purest sense, without having to worry about details of implementation.

However, the reality of *physical qubits* is quite different. Physical qubits can *decohere* or *lose quantum coherence*. When a qubit decoheres, the information on it is lost or corrupted. This is a principal motivator for quantum error correction schemes.

A.4 Gates

Qubits can be manipulated by quantum gates. Just as qubits are described by vectors, gates are described by matrices. A simple example is the Pauli X gate, which is given below:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

This is often called the “quantum not” operation, as it behaves like a not gate on basis states; taking $|0\rangle$ to $|1\rangle$ and vice versa. Another important gate is the *Hadamard* gate, shown below:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

The Hadamard gate is a foundational component in many quantum algorithms, as it is the simplest way to induce a superposition. When applied to $|0\rangle$, it creates a state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. When this state is measured, we get outcomes of zero or one with equal probability.

Multi-qubit gates also exist and we will give special attention to a class of gates called *controlled gates*. These gates apply a particular operator to a qubit *if and only if* a second “control” bit is set. Here for instance is the CNOT (sometimes CX) gate:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

This applies X to the second bit in a pair, if and only if the first bit is $|1\rangle$. It’s important to note that controlled gates maintain coherence; if the control bit is in some non-basis state, controlled gates do not measure and collapse the state. Rather, the operation is applied in a manner that propagates the amplitudes across the other qubit.

For example, let us consider apply CX to $|+\rangle|0\rangle$:

$$\text{CX}(|+\rangle \otimes |0\rangle) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

In this example, the $|+\rangle$ state is not measured and collapsed. Instead, after the gate application both qubits enter a superposition state. In this case, we find that both must agree — and thus we have a non-separable state.

Gates must maintain the property that the probability across all measurement outcomes is 1. All quantum gates must therefore be *unitary operators*. For every gate, the inverse is its *conjugate transpose*. This has other (potentially surprising) results for quantum information theory. For instance, all unitary operators are *reversible*. Therefore, all quantum computations formed of these gates must be reversible — the notable exception being measurement. In general, an n -bit quantum gate is a unitary operator. This usually manifests as a square ($2^n \times 2^n$), complex-valued unitary matrix.

A.5 Circuits

A single gate is hardly useful. To build larger programs, we must combine them, and we can do so in two different ways. *Sequencing* can be used to apply two gates to the same qubit. *Tensoring* can be used to apply gates to different qubits in parallel. By tensoring with the identity gate, we can also apply a gate to just one qubit in a larger collection.

Both of these ideas are underpinned by simple matrix operations. Two gates can be sequenced by multiplying them together. Note that the ordering in the multiplication is reverse: if we want to apply A then B, then the overall gate would be BA. Tensoring two gates together is computed by taking the Kronecker product of the two.

There is a natural visual interpretation for these operations as *quantum circuits*, as in Fig. 3. The visual interpretation of circuits make them very appealing — they are a common medium for discussing quantum algorithms and processes. However, there are significant differences between classical and quantum circuits. For example, in quantum circuits, wires must always have a fan-out of 1 — corresponding to the linearity of quantum information. The wires in a quantum circuit do not correspond to real wires. Instead, they indicate *data-flow* between gates in the circuit. The implementation is irrelevant — a detail to be abstracted away.

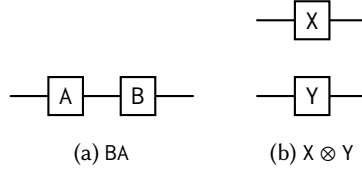


Fig. 3. Circuit representation of sequencing (3a) and tensoring (3b)

Still, there are limitations to the circuit model; by nature, circuits are direct representations of straight-line code. They do not model looping or recursive control flow structures — structures inherent to generalised computation.

Given that circuits are one of the principal ways we reason about quantum computing, it should be no surprise that they have had a significant impact on our approaches to QPL. Early approaches to QPL maintained the “flatness” of circuit code, to their detriment. As we shall see, flat codes are not as expressive as non-flat alternatives, and it led to a trend of algorithms being given fixed problem size descriptions, which is often less intuitive.

Modern approaches therefore break, at least somewhat, with the circuit model, to allow more natural, general descriptions of algorithms. The challenge to the QPL designer is to maintain the intuitive nature of circuits without sacrificing expressiveness.

B Example Programs

In this appendix we present the example programs we reference in the rest of this paper. We have already discussed quantum teleportation, in §2.4.1. Here, we present examples for each of the other points in the lattice. We include the pseudocode presentation of quantum teleportation again in 12 for completeness.

Listing 12. Quantum Teleportation (QC)

```
// @QC

fn bell_pair() {
```



```

    (a, b) <- fresh
    H a
    CX a b
    return (a, b)
}

fn teleport(q) {
    (a, b) <- bell_pair()
    CX q a
    H q

    [q', a'] = M [q, a]

    if (a') { X b }
    if (q') { Z b }
}

```

B.1 Deutsch's Algorithm

Consider a function on bits $f : \mathbb{B} \rightarrow \mathbb{B}$. There are four such possible functions: Id, Not, Const1, Const0. Two of these are balanced — producing 0 and 1 equally often (Id and Not). Two of these are constant — producing only 0 or only 1 (the two Const functions).

On a classical computer, determining if a one-bit function is balanced or constant requires 2 queries. Deutsch's algorithm on a quantum computer is able to answer in this question with only 1 query.

More generally, we can consider a function on n bits, $f : \mathbb{B}^n \rightarrow \mathbb{B}$. A balanced function on n bits produces 1 or 0 exactly half of the time each. A constant function produces only 1 or 0. A classical computer can only determine if a classical function is balanced or constant with $O(2^n)$ queries. Whereas the n -bit analogue of deutsch's algorithm (the Deutsch-Jozsa algorithm) can determine this in $O(1)$ time.

Deutsch's algorithm is usually of interest because it was the first algorithm that demonstrated quantum supremacy. We present it here because the algorithm requires no control flow — it can be described in a totally flat manner. Thus it sits at Pt. F. Listing 13 presents a Deutsch's algorithm in a quantum pseudocode.

Listing 13. Deutsch's algorithm (Flat)

```

// @Flat

// Construct the function to test
// in this example, we use const 1
fn const1(a, b) {
    X b
}

fn deutsch() {
    a, b <- fresh
    X b
    H a b
}

```

```

    const1(a, b)
    H a

    return M a
}

```

B.2 Parametric Deutsch's Algorithm

In principle, the *parametric* version of deutsch's algorithm we present is identical to the one we have already discussed. The key difference between this program and the one above is that the choice of function can be made at runtime. This is a somewhat contrived example, but the point is not to demonstrate a useful quantum algorithm. Rather, we want to find a simple program which occupies **Pt. CC**. Listing 14 presents a parametric version of Deutsch's algorithm.

Listing 14. Parametric Deutsch's algorithm (CC)

```

// @CC

// First we define the 4 possible 1-bit functions:
fn id(a,b) { CX a b }

fn not(a,b) { X a; CX a b }

fn const0(a,b) {}

fn const1(a,b) { X b }

// And then the actual implementation
// Note that 'choice' is only known at runtime,
// so this cannot be flattened any further
fn deutsch(choice: Nat){
  a, b <- fresh
  X b
  H a b
  match choice with
    0 -> { id(a,b) }
    1 -> { not(a,b) }
    2 -> { const0(a,b) }
    _ -> { const1(a,b) }

  H a

  return M a
}

```

B.3 n -bit-rand

The principle underpinning the n -bit-rand example is very simple: we repeatedly create and measure a $|+\rangle$ state in order to generate a random number. The number is drawn from the discrete uniform distribution on 2^n elements. There are 2^n choices, each of which is chosen with equal probability. The same problem can be solved at a much lower cost on a classical computer. It is of interest to us because it presents a simple program at [Pt. CL](#).

Generating a random number of some fixed bit width n can be expressed by a program at [Pt. F](#). One can unroll the loop in the program we are about to present to gain a fixed representation. However, in n -bit-rand, the bit width of the number to be generated is only known at runtime. This makes unrolling the loop statically impossible, and thus places the program at [Pt. CL](#). [Listing 15](#) presents n -bit-rand.

Listing 15. n -bit Random number (CL)

```
// @CL

// generates an n-bit random number
fn rand(n: Nat){
  num = bit[n]
  i = 0
  while (i < n) {
    a <- fresh
    H a
    num[i] = M a
    i += 1
  }

  return num
}
```

B.4 Geometric Distribution

This example was adapted from a program presented on the silq website. This is another random number generator, though there are important differences between this and n -bit-rand. This example samples not the uniform distribution, but the geometric distribution. That yields 1 with probability $\frac{1}{2}$, 2 with probability $\frac{1}{4}$, and in general, n with probability $\frac{1}{2^n}$ for $n > 0$.

It is, in a sense, a repeat-until-success algorithm. We repeatedly construct the $|+\rangle$ state and measure it, until we measure a true result. The program is an excellent example of mixed computing – requiring the interleaving of classical operations (such as adding one to a number) and quantum operations. We deliberately ignore some details (such as the representation of the number) as they are not relevant to the expressiveness hierarchy we are trying to explore. The data dependency between the measurement result and the condition of the loop is what places this program at [Pt. QL](#).

Listing 16. Sample Geometric Distribution (QL)

```
// @QL

// Sample the geometric distribution
// repeated runs will produce results whose probability
```

```
// distribution is a geometric distribution.

fn geometric() {
  count = 0
  x <- fresh
  do {
    H x
    res = M x
    count += 1
  } while (res)
  return count
}
```

C Other Code Samples

Here, we present implementations of the examples discussed in §B. Not all the languages are capable of representing all the examples.

C.1 Scaff & Scaff-qasm

For each example, we give both the scaffold code, and the output QASM it generates. Formally speaking, Scaffold and QASM sit at the flat static point of the lattice; Thus the only one of our sample algorithms they can implement is Deutsch's algorithm.

Listing 17. Deutsch's Algorithm in Scaffold

```
int main() {
  // Deutsch's algorithm
  cbit out[1];
  qbit qs[2];

  X ( qs[0] );
  H ( qs[0] );
  H ( qs[1] );
  X ( qs[1] );
  H ( qs[1] );

  out[0] = MeasZ ( qs[0] );

  return 0;
}
```

Listing 18. Deutsch's Algorithm in QASMH

```
-----QASM Generation Pass:

module main(){
  cbit out[1];
  qbit qs[2];
  X ( qs[0] );
```

```

        H ( qs[0] );
        H ( qs[1] );
        X ( qs[1] );
        H ( qs[1] );
        out[0] = MeasZ ( qs[0] );
    }

-----End of QASM generation

```

Although scaffold is a flat code, it can perform loop unrolling. Thus we have also included a sample for n -bit-*rand* — although the n in question must be hard coded at compile time.

Listing 19. n -bit (for fixed n) Random Generator in Scaffold

```

#define n 3

int main() {
    cbit out[n];
    qbit qs[1];

    for (int i = 0; i < n; i++ ) {
        H ( qs [0] );
        out[i] = MeasZ( qs[0] );
    }

    return 0;
}

```

Listing 20. n -bit (for fixed n) Random Generator in QASMH

```

-----QASM Generation Pass:

module main(){
    cbit out[3];
    qbit qs[1];
    H ( qs[0] );
    out[0] = MeasZ ( qs[0] );
    H ( qs[0] );
    out[1] = MeasZ ( qs[0] );
    H ( qs[0] );
    out[2] = MeasZ ( qs[0] );
}

-----End of QASM generation

```

Finally, we include some examples which demonstrate ScaffCC’s unintuitive compilation and silent failure.

Listing 21. Broken example — silently compiles to 0 instructions

```

int main() {

```

```

    qbit q;
    cbit c;

    H ( q );
    c = MeasZ ( q );

    return 0;
}

```

Listing 22. Output for previous example

```

-----QASM Generation Pass:

-----End of QASM generation

```

Listing 23. Broken example — silently compiles if statement out

```

int main() {
    qbit q[1];
    cbit c[1];

    H ( q[0] );

    c[0] = MeasZ ( q[0] );

    if (c[0]) {
        X ( q[0] );
    }

    return 0;
}

```

Listing 24. Output for previous example

```

-----QASM Generation Pass:

module main(){
    qbit q[1];
    cbit c[1];
    H ( q[0] );
    c[0] = MeasZ ( q[0] );
    X ( q[0] );
}

-----End of QASM generation

```

C.2 OpenQASM 2

Listing 25. Deutsch's Algorithm in OpenQASM 2

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[1];
x q[1];
h q[0];
h q[1];
x q[1];
h q[0];
measure q[0] -> c[0];
```

C.3 OpenQASM 3

Listing 26. Deutsch's Algorithm in OpenQASM 3

```
OPENQASM 3.0;
include "stdgates.inc";
bit[1] c;
qubit[2] q;
x q[1];
h q[0];
h q[1];
x q[1];
h q[0];
c[0] = measure q[0];
```

Listing 27. Parametric Deutsch's algorithm in OpenQASM 3

```
OPENQASM 3.0;
include "stdgates.inc";
bit[3] c0;
int switch_dummy;
qubit[2] q0;
x q0[1];
h q0[0];
h q0[1];
switch_dummy = c0;
switch (switch_dummy) {
  case 0 {
    cx q0[0], q0[1];
  }
  case 1 {
    x q0[0];
    cx q0[0], q0[1];
  }
  case 2 {
```



```

    }
    case 3 {
        x q0[1];
    }
}
h q0[0];
c0[2] = measure q0[0];

```

Listing 28. Quantum Teleportation in OpenQASM 3

```

OPENQASM 3.0;
include "stdgates.inc";
bit[2] c;
qubit[3] q;
h q[1];
cx q[1], q[2];
cx q[0], q[1];
h q[0];
c[0] = measure q[0];
c[1] = measure q[1];
if (c[0]) {
    x q[2];
}
if (c[1]) {
    z q[2];
}

```

Listing 29. Broken n -bit Random number in OpenQASM 3

```

OPENQASM 3.0;
include "stdgates.inc";
int[8] n = 10; // Should be determined by some external input
bit[10] c;
int[8] count;
bit[256] out
qubit[1] q;
while (count < n) {
    h q[0];
    c[count] = measure q[0]; // illegal; cannot index by a variable
                             // must be a compile time constant.
    count += 1;
}

```

Listing 30. n -bit Random number in OpenQASM 3

```

OPENQASM 3.0;
include "stdgates.inc";
int[8] n = 10; // Should be determined by some external input
int[16] c;

```

```

int[8] count;
qubit[1] q;
bit [1] cond;
while (count < n) {
    h q[0];
    cond[0] = measure q[0]
    if (cond[0]) {
        c += 1
    }
    c *= 2
    count += 1;
}

```

Listing 31. Sample Geometric Distribution in OpenQASM 3

```

OPENQASM 3.0;
include "stdgates.inc";
bit[1] c;
int[32] b = 0;
qubit[1] q;
while (!c[0]) {
    h q[0];
    c[0] = measure q[0];
    b += 1;
}

```

C.4 Quil

Listing 32. Deutsch's algorithm in Quil

```

DECLARE ro BIT[1]
X 1
H 0
H 1
X 1
H 0
MEASURE 0 ro[0]

```

Listing 33. Parametric Deutsch's algorithm in Quil

```

DECLARE choice INTEGER[1]
DECLARE scratch BIT[1]
DECLARE ro BIT[1]
X 1
H 0
H 1
EQ scratch[0] choice[0] 0
JUMP-WHEN @THEN_0 scratch[0]
JUMP @END_0

```

```

LABEL @THEN_0
CNOT 0 1
LABEL @END_0
EQ scratch[0] choice[0] 1
JUMP-WHEN @THEN_1 scratch[0]
JUMP @END_1
LABEL @THEN_1
X 0
CNOT 0 1
LABEL @END_1
EQ scratch[0] choice[0] 2
JUMP-WHEN @THEN_2 scratch[0]
JUMP @END_2
LABEL @THEN_2
LABEL @END_2
GT scratch[0] choice[0] 2
JUMP-WHEN @THEN_3 scratch[0]
JUMP @END_3
LABEL @THEN_3
X 1
LABEL @END_3
H 0
MEASURE 0 ro[0]

```

Listing 34. Quantum Teleportation in Quil

```

DECLARE results BIT[2]
H 0
CNOT 0 1
CNOT 3 0
H 3
MEASURE 3 results[0]
MEASURE 0 results[1]
JUMP-WHEN @THEN_0 results[0]
JUMP @END_0
LABEL @THEN_0
X 1
LABEL @END_0
JUMP-WHEN @THEN_1 results[1]
JUMP @END_1
LABEL @THEN_1
Z 1
LABEL @END_1

```

Listing 35. n -bit Random number in Quil

```

DECLARE n INTEGER[1]
DECLARE num INTEGER[1]
DECLARE scratch BIT[1]

```

```

DECLARE res BIT[1]
MOVE num[0] 0
LABEL @START_0
JUMP-UNLESS @END_0 scratch[0]
H 0
MEASURE 0 res[0]
MUL num[0] 2
JUMP-WHEN @THEN_0 res[0]
JUMP @END_1
LABEL @THEN_0
ADD num[0] 1
LABEL @END_1
ADD n[0] 1
EQ scratch[0] n[0] n[0]
NOT scratch[0]
JUMP @START_0
LABEL @END_0

```

Listing 36. Sample Geometric Distribution in Quil

```

DECLARE flag_register BIT[1]
DECLARE counter INTEGER[1]
MOVE flag_register[0] 1
LABEL @START_0
JUMP-UNLESS @END_0 flag_register[0]
ADD counter[0] 1
X 0
H 0
MEASURE 0 flag_register[0]
JUMP @START_0
LABEL @END_0

```

C.5 Qiskit

Listing 37. Deutsch's algorithm in Qiskit

```

from qiskit import QuantumCircuit

def deutsch():
    circuit = QuantumCircuit(2, 1)
    circuit.x(1)
    circuit.h(0)
    circuit.h(1)

    # const1 on second qubit
    circuit.x(1)

    circuit.h(0)
    circuit.measure(0, 0)

```

```
return circuit
```

Listing 38. Parametric Deutsch's algorithm in Qiskit

```
from qiskit import QuantumCircuit, QuantumRegister,
    ClassicalRegister

def qId(circ):
    circ.cx(0, 1)

def qNot(circ):
    circ.x(0)
    circ.cx(0, 1)

def qConst0(circ):
    pass

def qConst1(circ):
    circ.x(1)

def parametricDeutsch():
    creg = ClassicalRegister(3)
    qreg = QuantumRegister(2)
    circuit = QuantumCircuit(qreg, creg)
    circuit.x(1)
    circuit.h(0)
    circuit.h(1)

    with circuit.switch(creg) as case:
        with case(0):
            qId(circuit)
        with case(1):
            qNot(circuit)
        with case(2):
            qConst0(circuit)
        with case(3):
            qConst1(circuit)

    circuit.h(0)
    circuit.measure(0, 2)
    return circuit
```

Listing 39. Quantum Teleportation in Qiskit

```
from qiskit import QuantumCircuit

# We can't write an allocator;
# instead we have a function that puts 2 qubits
```

```

# into an entangled superposition
def entangle(circuit, a, b):
    circuit.h(a)
    circuit.cx(a, b)

# 0 is interesting bit
# 1 is partner on sender
# 2 is partner on receiver
def teleport():
    circuit = QuantumCircuit(3, 2)

    entangle(circuit, 1, 2)

    circuit.cx(0, 1)
    circuit.h(0)

    circuit.measure(0,0)
    circuit.measure(1,1)

    with circuit.if_test((0,1)):
        circuit.x(2)

    with circuit.if_test((1,1)):
        circuit.z(2)

    return circuit

```

As noted in the paper, Qiskit cannot represent either the n -bit-rand or geometric algorithms. These are therefore omitted.

C.6 Pyquil

Listing 40. Deutsch’s algorithm in Pyquil

```

from pyquil import Program
from pyquil.gates import *

# Define qubit names, for convenience
a = 0
b = 1

def const1(a, b):
    return X(b)

def deutsch():
    p = Program()
    p += X(b)
    p += H(a)
    p += H(b)

```

```

    p += const1(a, b)
    p += H(a)

    ro = p.declare('ro', 'BIT', 1)

    p += MEASURE(a, ro[0])

    return p

```

Listing 41. Parametric Deutsch's algorithm in Pyquil

```

from pyquil import Program
from pyquil.gates import *

# Define qubit names, for convenience
a = 0
b = 1

def qId(a,b):
    return CNOT(a, b)

def qNot(a,b):
    return Program() + X(a) + CNOT(a, b)

def const0(a,b):
    return Program()

def const1(a, b):
    return X(b)

def parametricDeutsch():
    p = Program()
    choice = p.declare('choice', 'INTEGER')
    scratch = p.declare('scratch', 'BIT', 1)

    p += X(b)
    p += H(a)
    p += H(b)

    p += EQ(scratch[0], choice, 0)
    p.if_then(scratch[0], qId(a,b))

    p += EQ(scratch[0], choice, 1)
    p.if_then(scratch[0], qNot(a,b))

    p += EQ(scratch[0], choice, 2)

```

```

p.if_then(scratch[0], const0(a,b))

p += GT(scratch[0], choice, 2)
p.if_then(scratch[0], const1(a,b))

p += H(a)

ro = p.declare('ro', 'BIT', 1)

p += MEASURE(a, ro[0])
p.resolve_placeholders()
return p

```

Listing 42. Quantum Teleportation in Pyquil

```

from pyquil import Program
from pyquil.gates import *

a = 0
b = 1
q = 3

def teleport():
    p = Program()

    p += H(a)
    p += CNOT(a, b)

    p += CNOT(q, a)
    p += H(q)

    results = p.declare('results', 'BIT', 2)
    p += MEASURE(q, results[0])
    p += MEASURE(a, results[1])

    p.if_then(results[0], X(b))
    p.if_then(results[1], Z(b))

    p.resolve_placeholders()
    return p

```

Listing 43. n -bit Random number in Pyquil

```

from pyquil import Program
from pyquil.gates import *

def rand():
    prog = Program()

```



```

n = prog.declare('n', 'INTEGER')

num = prog.declare('num', 'INTEGER')
prog += MOVE(num, 0)

count = prog.declare('n', 'INTEGER')
scratch = prog.declare('scratch', 'BIT', 1)

def loopCondition():
    return Program() + EQ(scratch, count, n) + NOT(scratch)

loop_body = Program()
res = loop_body.declare('res', 'BIT', 1)
loop_body += H(0)
loop_body += MEASURE(0, res)
loop_body += MUL(num, 2)
loop_body.if_then(res, ADD(num, 1))

loop_body += ADD(count, 1)
loop_body += loopCondition()

prog.while_do(scratch, loop_body)

prog.resolve_label_placeholders()
return prog

```

Listing 44. Sample Geometric Distribution in Pyquil

```

from pyquil import Program
from pyquil.gates import *

# Initialize the Program and declare a 1 bit memory space for our
# boolean flag
def geometric():
    prog = Program()
    flag_register = prog.declare('flag_register', 'BIT')
    counter = prog.declare('counter', 'INTEGER')

    # Set the initial flag value to 1
    prog += MOVE(flag_register, 1)

    # Define the body of the loop with a new Program
    loop_body = Program()
    loop_body += ADD(counter, 1)
    loop_body += X(0)
    loop_body += H(0)

```

```

loop_body += MEASURE(0, flag_register)

# Run loop_body in a loop until flag_register is 0
prog.while_do(flag_register, loop_body)
prog.resolve_label_placeholders()

return prog

```

C.7 Cirq

Listing 45. Deutsch's algorithm in Cirq

```

import cirq

def qId(a, b):
    return [cirq.CX(a, b)]

def qNot(a, b):
    return [cirq.CX(a, b), cirq.X(b)]

def const1(a, b):
    return [cirq.X(b)]

def const0(a, b):
    return []

def deutsch():
    a, b = cirq.LineQubit.range(2)
    c = cirq.Circuit([
        cirq.X(b),
        cirq.H(a),
        cirq.H(b),
        *qId(a, b),
        cirq.H(a),
        cirq.measure(a, key="result")
    ])
    return c

```

C.8 Quipper

Listing 46. Deutsch's algorithm in Quipper

```

import Quipper

const1 :: Qubit -> Qubit -> Circ (Qubit, Qubit)
const1 p q = do
  q <- gate_X q
  return (p, q)

```

```
deutsch :: Circ Bit
deutsch = do
  a <- qinit True >=> hadamard
  b <- qinit False >=> hadamard
  (a, b) <- const1 a b
  res <- hadamard a >=> measure
  qdiscard b
  return res

main :: IO ()
main = print_generic PDF deutsch
```

Listing 47. Parametric Deutsch's algorithm in Quipper

```
import Quipper

qId :: Qubit -> Qubit -> Circ (Qubit, Qubit)
qId a b = do
  a <- gate_X a `controlled` b
  return (a, b)

qNot :: Qubit -> Qubit -> Circ (Qubit, Qubit)
qNot a b = do
  a <- gate_X a
  b <- gate_X b `controlled` a
  return (a, b)

const0 :: Qubit -> Qubit -> Circ (Qubit, Qubit)
const0 a b = return (a, b)

const1 :: Qubit -> Qubit -> Circ (Qubit, Qubit)
const1 a b = do
  b <- gate_X b
  return (a, b)

mkNot :: Bit -> Circ (Bit, Bit)
mkNot a = do
  b <- cinit True
  b <- cnot b `controlled` a
  return (a, b)

applyChoice :: (Bit, Bit) -> Qubit -> Qubit -> Circ (Qubit, Qubit)
)
applyChoice (x, y) a b = do
  (x, nx) <- mkNot x
  (y, ny) <- mkNot y
  choice0 <- cgate_and [nx, ny]
  choice1 <- cgate_and [x, ny]
```

```

choice2 <- cgate_and [nx, y]
choice3 <- cgate_and [x, y]

(a, b) <- qId a b `controlled` choice0
(a, b) <- qNot a b `controlled` choice1
(a, b) <- const0 a b `controlled` choice2
(a, b) <- const1 a b `controlled` choice3

cdiscard (x, nx, y, ny, choice0, choice1, choice2, choice3)
return (a, b)

deutsch :: Bit -> Bit -> Circ Bit
deutsch x y = do
  (a, b) <- qinit (True, False) >>= map_hadamard
  (a, b) <- applyChoice (x, y) a b
  res <- hadamard a >>= measure
  qdiscard b
  return res

main :: IO ()
main = print_simple PDF deutsch

```

Listing 48. Quantum Teleportation in Quipper

```

import Quipper

bellPair :: Circ (Qubit, Qubit)
bellPair = do
  a <- qinit false >>= hadamard
  b <- qinit false
  b <- gate_X b `controlled` a
  return (a, b)

teleport :: Qubit -> Circ Qubit
teleport q = do
  (a, b) <- bellPair
  q <- gate_X q `controlled` a >>= hadamard >>= measure
  a <- measure a
  b <- gate_X b `controlled` a
  b <- gate_Z b `controlled` q
  cdiscard (a, q)
  return b

main :: IO ()
main = print_generic PDF $ teleport (qinit False)

```

Listing 49. *n*-bit Random number in Quipper

```

import Quipper

```

```

nBitRand :: Int -> Circ [Bit]
nBitRand 0 = []
nBitRand count = do
  newBit <- qinit False >>= hadamard >>= measure
  otherBits <- nBitRand $ count - 1
  return newBit : otherBits

```

Listing 50. *n*-bit Random number with dynamic lifting in Quipper

```

import Quipper

type CInt = [Bit]

cIntZero :: CInt -> Circ Bit
cIntZero [] = cinit True
cIntZero (x : xs) =
  if x == cinit True
    then cinit False
    else cIntZero xs

cIntDec :: CInt -> Circ CInt
cIntDec [] = []
cIntDec (bit : rest) = do
  restIsZero <- cIntZero rest
  bit <- cnot bit `controlled` restIsZero
  rest <- cIntDec rest
  return (bit : rest)

nBitRand :: [Bit] -> Circ [Bit]
nBitRand count = do
  isZero <- cIntZero rest >>= dynamic_lift

  if isZero
    then return []
    else do
      count <- cIntDec count
      newBit <- qinit False >>= hadamard >>= measure
      otherBits <- nBitRand count
      return newBit : otherBits

```

Listing 51. Sample Geometric Distribution in Quipper

```

import Quipper

geometricInner :: Int -> Circ Int
geometricInner count = do
  res <- qinit False >>= hadamard >>= measure >>= dynamic_lift
  if res

```

```

    then geometricInner (count + 1)
    else return count

geometric :: () -> Circ Int
geometric _ = geometricInner 1

main :: IO ()
main = print_generic PDF geometric

```

C.9 Silq

Listing 52. Deutsch's algorithm in Silq

```

def const1(a :  $\mathbb{B}$ , b :  $\mathbb{B}$ ) {
    b := X(b);

    return (a,b);
}

def deutsch() {
    a := false :  $\mathbb{B}$ ;
    b := false :  $\mathbb{B}$ ;

    a := X(a);
    a := H(a);
    b := H(b);

    (a, b) := const1(a, b);

    a := H(a);
    res := measure(a);

    // Manually discard b
    measure(b);

    return res;
}

```

Listing 53. Parametric Deutsch's algorithm in Silq

```

def CX(a:  $\mathbb{B}$ , b:  $\mathbb{B}$ ) {
    if (a) {
        b := X(b)
    }
    return (a, b)
}

// First we define the 4 possible 1-bit functions:
def id(a:  $\mathbb{B}$ , b:  $\mathbb{B}$ ) { return CX(a, b); }

```

```

def not(a:  $\mathbb{B}$ , b:  $\mathbb{B}$ ) {
    a := X(a);
    return CX(a, b);
}

def const0(a:  $\mathbb{B}$ , b:  $\mathbb{B}$ ) { return (a, b); }

def const1(a:  $\mathbb{B}$ , b:  $\mathbb{B}$ ) {
    b := X(b);
    return (a, b);
}

def deutsch(f: !( $\mathbb{B}^2 \rightarrow \mathbb{B}^2$ ) ) {
    a := false :  $\mathbb{B}$ ;
    b := false :  $\mathbb{B}$ ;

    a := X(a);
    a := H(a);
    b := H(b);

    (a, b) := f(a, b);

    a := H(a);
    res := measure(a);

    // Manually discard b
    measure(b);

    return res;
}

def parametric(x: ! $\mathbb{N}$ ) {
    if (x == 0) { deutsch(id); }
    if (x == 1) { deutsch(not); }
    if (x == 2) { deutsch(const0); }
    else      { deutsch(const1); }
}

```

Listing 54. Quantum Teleportation in Silq

```

def bell_pair () {
    a := false :  $\mathbb{B}$ ;
    b := false :  $\mathbb{B}$ ;
    a := H(a);

    if a {

```

```

        b := X(b);
    }

    return (a, b);
}

def teleport(q:  $\mathbb{B}$ ) {
    (a, b) := bell_pair();
    if q { a := X(a); }
    q := H(q);

    q' := measure(q);
    a' := measure(a);

    if (a') { b := X(b) }
    if (q') { b := Z(b) }

    return b
}

```

Listing 55. n -bit Random number in Silq

```

def rand(n: ! $\mathbb{N}$ ) {
    num := 0;
    i := 0;

    while (i < n) {
        res := measure(H(false));
        if res {
            num += 1
        }
        num *= 2;
        i += 1;
    }

    return num;
}

```

Listing 56. Sample Geometric Distribution in Silq

```

def geometric():! $\mathbb{N}$ {
    count := 0;
    ok := true;
    while ok {
        count += 1;
        ok = measure(H(false));
    }
    return count;
}

```


Here, we also present a program which Silq's type checker rejects. This is a silq implementation of the program which breaks stack based allocation. Since b is not consumed, the program is rejected.

Listing 57. Stack unsafe in Silq

```
def foo(a:  $\mathbb{B}$ ) {  
    b := 0 :  $\mathbb{B}$ ;  
    b := H(b);  
    if b {  
        a := X(a);  
    }  
    return a;  
}  
  
def main() {  
    a := 0 :  $\mathbb{B}$ ;  
    a := foo(a);  
    return measure(a);  
}
```